VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF INDUSTRIAL ENGINEERING AND MANAGEMENT

MULTI-OBJECTIVE MULTI-AGENT DEEP REINFORCEMENT LEARNING FOR MULTI-ECHELON INVENTORY SYSTEMS UNDER DEMAND UNCERTAINTY

Submitted in partial fulfillment of the requirements for the Degree of Bachelor of Engineering in Logistics and Supply Chain Management/

Student: Duong Nguyen Tung Phuong
ID: IELSIU21223
Thesis Advisor: ASC. PROF. DR. NGUYEN VAN HOP

Ho Chi Minh City, Vietnam

January 2026

MULTI-OBJECTIVE MULTI-AGENT DEEP REINFORCEMENT LEARNING FOR MULTI-ECHELON INVENTORY SYSTEMS UNDER DEMAND UNCERTAINTY

By

DUONG NGUYEN TUNG PHUONG

Submitted in partial fulfillment of the requirements for the Degree of Bachelor of Engineering in Logistics and Supply Chain Management

International University, Ho Chi Minh City
January 2026

Signature of Student: _____

Duong Nguyen Tung Phuong

Certified by _____

Assoc. Prof. Dr. Nguyen Van Hop
Thesis Advisor

Approved by _____

Assoc. Prof. Dr. Nguyen Van Hop
Dean of IEM school

Abstract

Multi-echelon inventory management under stochastic demand requires balancing adaptive decision-making with rigid operational constraints, a challenge often unmet by pure Deep Reinforcement Learning (DRL) due to practically infeasible lot-sizing. To address this, this thesis proposes a Hybrid Neuro-Symbolic Framework that integrates Heterogeneous-Agent Proximal Policy Optimization (HAPPO) for strategic ordering with a Genetic Algorithm (GA) for tactical refinement. By employing a hierarchical control strategy, the system ensures strict constraint satisfaction while systematically exploring the cost-service trade-off. Extensive experiments on serial and network topologies demonstrate that the Hybrid model significantly outperforms the Pure HAPPO baseline, achieving Fill Rate improvements exceeding 20%. Crucially, the framework exhibits superior scalability, successfully maintaining stability and cost efficiency in complex network configurations where the baseline suffered catastrophic performance degradation. Furthermore, the system demonstrates a novel "upstream dampening" phenomenon, effectively reversing classical bullwhip amplification and confirming the hybrid approach as a resilient solution for modern inventory optimization.

Keywords: Multi-Agent Reinforcement Learning (MARL), Multi-Echelon Inventory Optimization, Genetic Algorithm, Neuro-Symbolic AI, Bullwhip Effect, HAPPO.

Acknowledgements

I would like to express my deepest gratitude to my thesis advisor, Assoc. Prof. Dr. Nguyen Van Hop, for his invaluable guidance, insightful feedback, and continuous support throughout the development of this research. His expertise and encouragement have been instrumental in shaping both the direction and the quality of this study.

My sincere thanks also go to the faculty members and staff of the School of Industrial Engineering and Management, International University – Vietnam National University Ho Chi Minh City, for providing the academic foundation and learning environment that made this work possible.

I am deeply grateful to my family and friends for their encouragement, patience, and unwavering belief in my ability to complete this journey. Their support has been a constant source of motivation.

Finally, I would like to acknowledge all those who, directly or indirectly, contributed to the successful completion of this thesis.

Table of Content

Abstract i

Acknowledgements ii

List of Tables vii

List of Figures ix

List of Abbreviations x

List of Symbols xi

CHAPTER 1: INTRODUCTION 1

1.1. Backgrounds 1

1.2. Problem Statement 2

1.3. Objective of Study 3

1.4. Scope and Limitation 4

CHAPTER 2: RELATED WORKS 5

2.1. Overview of Key Concepts 5

2.1.1. Multi-Echelon Inventory Systems and Supply Chain Resilience 5

2.1.2. The Bullwhip Effect 5

2.1.3. The Lot-Sizing Problem 6

2.2. Related Works 7

2.2.1. Reinforcement Learning for Inventory Management 7

2.2.2. Multi-Agent Reinforcement Learning 7

2.2.3. Multi-Agent Approaches for Supply Chain 8

2.2.4. Hybrid and Constrained Approaches 9

2.2.5. Summary and Research Gaps 10

2.3. Key References & Candidate Solution Methods 11

2.3.1. HAPPO (Heterogeneous Agent PPO) 11

2.3.2. Genetic Algorithm for Lot-Sizing 11
2.3.3. Hybrid Planner Approach 12
2.3.4. Multi-Objective Optimization Framework 12
CHAPTER 3: METHODOLOGY 14
3.1. Approaches Comparison and Selection 14
3.1.1. Qualitative Comparison 14
3.1.2. Quantitative and Logical Justification 15
3.1.3. Selection Rationale 16
3.2. Proposed System Design or Proposed Solution Approach 16
3.2.1. System Architecture Overview 17
3.2.2. Multi-Objective Reward Formulation 18
3.2.3. Observation Space Design 19
3.2.4. Hybrid Refinement Mechanism 21
3.2.5. Learning Algorithm Framework 23
CHAPTER 4: SOLUTION DEVELOPMENT 25
4.1. Model / Prototype Solution 25
4.1.1. Mathematical Formulation 25
4.1.2. Neural Network Specifications 28
4.1.3. Algorithm Parameters 29
4.1.4. GA Lot-Sizing Parameters 30
4.2. Solution Development 30
4.2.1. Solution Approaches Overview 30
4.2.2. Neural Network Architecture 31
4.2.3. Training Algorithm 32
4.2.4. Hybrid Planner Module 35

4.2.5. Solution Comparison Framework 36
4.3. Environment Implementation 37
4.3.1. Serial Supply Chain Environment 37
4.3.2. Network Supply Chain Environment 40
4.3.3. Vectorized Training Architecture 44
CHAPTER 5: RESULT ANALYSIS 47
5.1. Experimental Design 47
5.1.1. Research Questions 47
5.1.2. Experimental Setup 48
5.1.3. Multi-Objective Optimization Framework 51
5.1.4. Evaluation Metrics 52
5.1.5. Statistical testing framework 55
5.2. Result on Serial (3-echelon chain) 59
5.2.1. Training Convergence 59
5.2.2. Pareto Frontier Analysis 60
5.2.3. Cost Performance 61
5.2.4. Service Level 62
5.2.5. Bullwhip Effect Analysis (Stability) 62
5.2.6. Statistical Tests 63
5.3. Result on Network (2×3 topology) 63
5.3.1. Training Convergence 63
5.3.2. Cost Performance 64
5.3.3. Bullwhip Effect 65
5.3.4. Service Level 65
5.3.5. Statistical Tests 66

5.4. Comparative analysis 66
5.4.1. Cross-Environment Comparison 66
5.4.2. Scalability Analysis 68
5.4.3. Sensitivity Analysis 68
5.4.4. Computational Efficiency 72
CHAPTER 6: CONCLUSIONS 74
6.1. Summary and Key Findings 74
6.2. Contributions 75
6.3. Limitations 77
6.4. Future Research Directions 77
6.5. Concluding Remarks 78
References 79

List of Tables

Table 2-1. Summary of Related Work 10
Table 3-1. Qualitative comparison of methodological approaches 14
Table 3-2. Pareto Weight Configurations for Multi-Objective Optimization ... 19
Table 3-3. Early Stopping and Learning Rate Adaptation Configuration 23
Table 4-1. Cost parameter across 3 echelons 27
Table 4-2. Multi-Objective Reward Component Specifications 27
Table 4-3. Actor Network Architecture details 28
Table 4-4. HAPPO Training Hyperparameters 29
Table 4-5. Pareto Weight Configurations 29
Table 4-6. Genetic Algorithm Configuration 30
Table 4-7. Network Specifications 31
Table 4-8. Early Stopping Configuration Parameters 34
Table 4-9. Comparative Analysis of Solution Approaches 36
Table 5-1. Summary of Research Hypotheses and Statistical Testing Framework 48
Table 5-2. Environment Parameters Common to Both Configurations 49
Table 5-3. Cost Parameters Across Supply Chain Echelons 49
Table 5-4. HAPPO Training Hyperparameters 50
Table 5-5. Pareto Weight Configurations 51
Table 5-6. Interpretation Guidelines for Cohen's d Effect Size 58
Table 5-7. Significance Level Notation 58
Table 5-8. Statistical Testing Framework Summary 59
Table 5-9. Performance Comparison Summary 60
Table 5-10. Pareto Frontier Results - Serial Topology 60
Table 5-11. Bullwhip Effect Comparison 62
Table 5-12. Training Convergence Network 64
Table 5-13. Cost Performance Comparison (Network Topology) 64
Table 5-14. Bullwhip Effect Analysis (Network) 65
Table 5-15. Service Level Comparison (Network) 65

Table 5-16. Cross-Environment Performance Summary.....66
Table 5-17. Scalability Metrics 68
Table 5-18. Pareto Weight Configurations 69
Table 5-19. Performance Across Pareto Configurations 70
Table 5-20. Bullwhip Effect by Echelon 71
Table 5-21. Cost Distribution by Echelon 71
Table 5-22. Recommended Configurations by Business Priority.....72
Table 5-23.Computational Efficiency Comparison.....72

List of Figures

Figure 3-1. Hybrid Neuro-Symbolic Control Loop 15
Figure 3-2. System Architecture Overview - Hierarchical Data Flow 17
Figure 3-3. HybridPlanner - Decision and Refinement Logic 21
Figure 3-4. CTDE Framework for Multi-Echelon Inventory Optimization 23
Figure 4-1. The proposed Hybrid-HAPPO training framework. 25
Figure 4-2.Serial supply chain topology with three echelons. Material flows downstream from Factory to Customer, while orders propagate upstream 38
Figure 4-3. Network supply chain topology with two parallel distribution channels (2×3 configuration) 42
Figure 4-4. Vectorized Training Architecture 45
Figure 5-1. Training convergence 60
Figure 5-2. Sensitivity Analysis - Reward Scalarization Logic 69
Figure 5-3. Pareto Frontier - Serial Topology 70
Figure 5-4. Bullwhip effect by agent - serial 71

List of Abbreviations

Abbreviation	Full Term
AGASOR	Adaptive Genetic Algorithm for Sequencing and Ordering Rules
AI	Artificial Intelligence
CTDE	Centralized Training with Decentralized Execution
DALSA	Demand Allocation and Lot-Sizing Algorithm
Dec-POMDP	Decentralized Partially Observable Markov Decision Process
DRL	Deep Reinforcement Learning
EOQ	Economic Order Quantity
FR	Fill Rate
GA	Genetic Algorithm
HARL	Heterogeneous-Agent Reinforcement Learning
HAPPO	Heterogeneous-Agent Proximal Policy Optimization
HATRPO	Heterogeneous-Agent Trust Region Policy Optimization
LSP	Lot-Sizing Problem
MARL	Multi-Agent Reinforcement Learning
MOQ	Minimum Order Quantity
MDP	Markov Decision Process
PPO	Proximal Policy Optimization
RL	Reinforcement Learning

List of Symbols

Sets & Indices

Symbol	Description
i	Index of agent / supply chain echelon
t	Time period index
N	Number of agents (echelons)
T	Planning horizon
$\mathcal{I}$	Set of agents

State, Action & Decision Variables

Symbol	Description
s_t	Global system state at time t
o_t^i	Local observation of agent i at time t
a_t^i	Action (ordering decision) of agent i at time t
q_t^i	Executed order quantity of agent i at time t
I_t^i	Inventory level of agent i at time t
B_t^i	Backlog of agent i at time t

Cost Parameters

Symbol	Description
h	Holding cost per unit
b_i	Backlog penalty cost for agent i
K	Fixed ordering cost
C_t^i	Total cost incurred by agent i at time t

Reinforcement Learning & Optimization

Symbol	Description
R_t^i	Reward of agent i at time t
R_i^{MO}	Multi-objective reward of agent i
α	Weight for cost minimization objective
β	Weight for service level maximization
γ	Discount factor in reinforcement learning

$\gamma_s c_{ale}$	Scaling factor for service reward
π_i	Policy of agent i
V	Value function
Q	Action-value function

Performance Metrics

Symbol	Description
FR_i	Fill rate of agent i
BW_i	Bullwhip measure at agent i
$\sigma^2(\cdot)$	Variance operator

CHAPTER 1: INTRODUCTION

1.1. Backgrounds

Effective supply chain management has become a critical factor in determining organizational competitiveness in the contemporary business environment. Among the various functions within supply chain management, inventory control stands out as particularly challenging due to its direct impact on both operational costs and customer satisfaction. When inventory decisions must be coordinated across multiple echelons of a supply chain, the complexity increases substantially, as actions taken at one facility directly affect the performance of others throughout the network.

The multi-echelon inventory problem has attracted considerable research attention since the foundational work of Clark and Scarf (1960), who established the theoretical basis for echelon stock policies in serial supply chains. Their work demonstrated that optimal policies could be decomposed across echelons under certain conditions, providing a tractable approach for simple configurations. However, as noted by Boute et al. (2022), the assumptions underlying classical analytical methods rarely hold in practice, where demand exhibits high variability, non-stationarity, and sudden disruptions that violate the independent and identically distributed assumptions of traditional models.

The bullwhip effect, first documented systematically by Lee et al. (1997), represents another persistent challenge in multi-echelon systems. This phenomenon, wherein order variability amplifies as it propagates upstream through the supply chain, leads to excessive inventory investments, production inefficiencies, and degraded service levels. Traditional mitigation strategies focus on information sharing and coordination mechanisms, yet implementing such strategies in decentralized supply chains remains difficult.

Recent advances in artificial intelligence, particularly deep reinforcement learning, offer promising alternatives for addressing these challenges. Reinforcement learning enables agents to learn effective decision policies through direct interaction with the environment, without requiring explicit demand models or predetermined decision rules. The success of deep reinforcement learning in complex sequential

decision problems, from game playing to robotic control, has motivated its application to operations management domains including inventory control (Gijsbrechts et al., 2022).

Multi-agent reinforcement learning extends these capabilities to settings with multiple decision makers, enabling the study of coordination and cooperation in decentralized systems. Liu et al. (2024) recently demonstrated the effectiveness of Heterogeneous-Agent Proximal Policy Optimization (HAPPO), developed by Kuba et al. (2022), for multi-echelon inventory management, showing that multi-agent approaches can outperform single-agent baselines while reducing the bullwhip effect through implicit information sharing during centralized training.

However, applying reinforcement learning to practical inventory problems introduces additional considerations. Learned policies may propose order quantities that violate operational constraints such as minimum lot sizes or capacity limitations. Furthermore, inventory managers typically face trade-offs between competing objectives, particularly cost minimization and service level maintenance, requiring methods that can systematically explore the spectrum of Pareto-optimal solutions.

1.2. Problem Statement

This thesis addresses the multi-echelon inventory management problem under stochastic demand, focusing on both serial and network supply chain configurations. The problem involves multiple agents, each responsible for ordering decisions at a distinct echelon, who must coordinate their actions to optimize system-wide performance while respecting operational constraints.

The specific challenges motivating this research are as follows. First, coordination among autonomous agents presents fundamental difficulties, as each agent observes only local information while its decisions affect partners throughout the supply chain. This partial observability complicates individual optimization and can lead to suboptimal system performance when agents act purely in their own interest.

Second, the bullwhip effect continues to plague multi-echelon systems, with demand signal distortion causing order variability to exceed demand variability at upstream echelons. Mitigating this effect requires mechanisms that enable agents to implicitly or explicitly share information about demand conditions.

Third, practical inventory management involves multiple, often conflicting objectives. Minimizing inventory holding costs may compromise service levels, while maintaining high fill rates typically requires larger safety stocks. Practitioners require methods that illuminate the trade-off frontier between these objectives rather than imposing a single predetermined balance.

Fourth, reinforcement learning policies, while adaptive, may propose decisions that violate lot-sizing constraints or other operational requirements. Bridging the gap between learned policies and implementable decisions necessitates hybrid approaches that preserve learning capability while ensuring constraint satisfaction.

1.3. Objective of Study

The primary objective of this thesis is to develop and evaluate a hybrid framework combining multi-agent deep reinforcement learning with metaheuristic optimization for multi-echelon inventory management. This framework aims to leverage the adaptive learning capabilities of reinforcement learning while ensuring that ordering decisions satisfy practical operational constraints.

The specific objectives of this research are:

1. To implement and adapt the Heterogeneous-Agent Proximal Policy Optimization algorithm for decentralized inventory control across multiple supply chain configurations, enabling agents with heterogeneous roles to develop coordinated ordering policies.
2. To design a hybrid refinement mechanism integrating genetic algorithm-based lot-sizing optimization with the reinforcement learning policy, ensuring that proposed order quantities respect operational constraints while maintaining policy adaptability.
3. To develop a multi-objective optimization framework with configurable weight parameters that enables systematic exploration of the Pareto frontier between cost efficiency and service level performance.
4. To evaluate the proposed hybrid approach through comprehensive experiments on serial and network supply chain configurations, comparing performance against baseline methods across multiple metrics including total cost, fill rate, and bullwhip effect.

5. To analyze the sensitivity of the hybrid approach to variations in demand volatility and cost parameters, providing insights into the conditions under which the proposed method offers the greatest advantages.

1.4. Scope and Limitation

This thesis focuses on periodic-review inventory systems with stochastic demand in discrete-time settings. The experimental evaluation encompasses two supply chain configurations: a three-echelon serial structure representing a traditional manufacturer-distributor-retailer chain, and a six-agent network structure comprising two parallel supply chains serving independent demand streams.

The methodological scope includes the implementation of HAPPO within the Centralized Training with Decentralized Execution paradigm, where agents share information through a centralized critic during training but execute policies based solely on local observations during deployment. The genetic algorithm refinement operates on a rolling horizon basis, adjusting the order quantities proposed by the learned policy to satisfy lot-sizing considerations.

The multi-objective exploration employs weighted scalarization with five distinct configurations spanning from pure cost focus to pure service focus, enabling characterization of the Pareto frontier without requiring more computationally intensive multi-objective optimization algorithms.

Several aspects fall outside the scope of this thesis. Demand forecasting mechanisms are not explicitly integrated; the reinforcement learning agents implicitly learn demand patterns through experience rather than utilizing separate forecasting models. Mixed-integer linear programming solvers, while representing a promising direction for constraint handling, are not incorporated in the current framework. The experimental evaluation considers single-product inventory systems; multi-product extensions with substitution effects or shared resources are not addressed. Finally, capacity constraints at individual facilities are not modeled; the environments assume sufficient production or procurement capacity to fulfill any feasible order quantity.

CHAPTER 2: RELATED WORKS

2.1. Overview of Key Concepts

2.1.1. Multi-Echelon Inventory Systems and Supply Chain Resilience

Multi-echelon inventory systems comprise interconnected stages where inventory buffers decouple operations and absorb demand variability. The foundational analysis by Clark and Scarf (1960) established that echelon-based stock policies achieve optimality for serial systems under restrictive assumptions such as stationary demand, linear holding expenses, and complete information. Subsequent extensions by Federgruen and Zipkin (1984) accommodated more general network structures but retained assumptions that are rarely satisfied in practice.

The bullwhip effect, which refers to order variability amplification propagating upstream, exemplifies the coordination failures endemic to decentralized supply chains (Lee et al., 1997). This amplification stems from demand signal processing, order batching, and shortage gaming, resulting in inventory carrying burdens estimated at 20–30% above necessary levels (Metters, 1997). Traditional analytical methods address such phenomena primarily under idealized conditions and therefore provide limited guidance when distributional assumptions fail.

The COVID-19 pandemic exposed critical vulnerabilities in lean, globally distributed networks that were optimized solely for fiscal efficiency (Ivanov and Dolgui, 2020). As a result, supply chain resilience, defined as the capacity to anticipate, respond to, and recover from disruptions, has emerged as a co-equal objective alongside cost minimization. Resilient inventory positioning requires maintaining safety stocks that purely efficiency-focused frameworks would deem excessive, which in turn creates tension between short-term performance and long-term robustness. This inherent tension motivates the adoption of approaches capable of balancing multiple competing objectives rather than optimizing a single metric.

2.1.2. The Bullwhip Effect

The bullwhip effect refers to the phenomenon whereby order variability amplifies as it propagates upstream through a supply chain. Lee et al. (1997) identified four primary causes: demand signal processing, rationing and shortage gaming, order

batching, and price fluctuations. The consequences include excessive inventory investment, poor customer service, inefficient capacity utilization, and increased costs throughout the supply chain.

Mitigation strategies traditionally focus on information sharing, lead time reduction, and coordination mechanisms. However, implementing such strategies in decentralized supply chains where participants act autonomously remains challenging. Recent research has explored how learning-based approaches might implicitly achieve coordination through shared training experiences, potentially offering new avenues for bullwhip mitigation.

2.1.3. The Lot-Sizing Problem

The lot-sizing problem (LSP) determines order timing and quantities to satisfy demand over a planning horizon while minimizing aggregate expenditure. Wagner and Whitin (1958) established that optimal solutions exhibit the zero-inventory property, whereby orders occur only when stock is depleted and quantities match cumulative demand over integer future periods. This structural insight underpins numerous heuristic procedures, including Silver-Meal, Part Period Balancing, and Least Unit Cost, which exploit zero-inventory properties to achieve computational efficiency.

Computational complexity escalates dramatically under realistic operational constraints. Capacitated lot-sizing is NP-hard (Bitran and Yanasse, 1982), and multi-level formulations with dependent demand relationships further compound intractability. Minimum order quantities, truck capacities, and supplier-specific lead times introduce hard constraints that pure neural network approaches struggle to satisfy consistently, which constitutes a critical impediment to industrial deployment where any constraint violation renders solutions infeasible.

Metaheuristic approaches have been developed to address the computational complexity of lot-sizing problems. Nguyen Van Hop (2006) proposed the Adaptive Genetic Algorithm for Sequencing and Ordering Rules (AGASOR) for scheduling problems with setup considerations. Subsequently, Nguyen Van Hop (2010) introduced the Demand Allocation and Lot-Sizing Algorithm (DALSA), a two-phase heuristic that first allocates demand to suppliers and then determines optimal lot sizes considering

capacity and fixed costs. These heuristics demonstrate the applicability of evolutionary computation to discrete optimization problems in supply chain contexts.

2.2. Related Works

2.2.1. Reinforcement Learning for Inventory Management

The application of reinforcement learning to inventory control has gained substantial momentum in recent years. Boute et al. (2022) provided a comprehensive roadmap identifying key challenges and opportunities, noting that deep reinforcement learning offers potential advantages over traditional methods by eliminating the need for explicit demand models and enabling adaptation to complex, non-stationary environments.

Gijsbrechts et al. (2022) conducted an extensive empirical study comparing deep reinforcement learning approaches against optimal and heuristic baselines across lost sales, dual-sourcing, and multi-echelon problem settings. Their results demonstrated that properly configured reinforcement learning agents could match or exceed the performance of well-tuned heuristics, particularly in settings where analytical solutions are unavailable.

De Moor et al. (2022) investigated reward shaping techniques to improve the sample efficiency of deep reinforcement learning in perishable inventory management. Their work highlighted the importance of reward function design in guiding learning toward desirable policies and avoiding suboptimal local minima.

Dehaybe et al. (2024) addressed inventory optimization under non-stationary uncertain demand, demonstrating that deep reinforcement learning could effectively handle demand distributions that change over time without requiring explicit change-point detection or model updating.

Qi et al. (2023) presented a practical end-to-end inventory management model deployed at Alibaba, demonstrating the feasibility of deep learning approaches in large-scale industrial applications. Their work emphasized the importance of integrating domain knowledge with learning algorithms to achieve robust performance.

2.2.2. Multi-Agent Reinforcement Learning

Multi-agent reinforcement learning extends single-agent methods to settings with multiple interacting decision makers. The field has developed rapidly, with applications

spanning cooperative, competitive, and mixed settings across domains including robotics, game playing, and autonomous systems.

In cooperative multi-agent settings relevant to supply chain management, agents share a common objective but must coordinate their actions based on partial observations. The Centralized Training with Decentralized Execution paradigm has emerged as a practical approach, where agents access global information during training but execute policies based only on local observations during deployment.

Kuba et al. (2022) introduced Heterogeneous-Agent Trust Region Policy Optimization (HATRPO) and Heterogeneous-Agent Proximal Policy Optimization (HAPPO), addressing limitations of existing methods that rely on parameter sharing among agents. Their algorithms provide theoretical guarantees of monotonic improvement and convergence while accommodating agents with different observation and action spaces. The sequential update scheme central to these algorithms ensures that policy updates across agents remain coordinated.

Zhong et al. (2024) extended this line of work by developing the Heterogeneous-Agent Reinforcement Learning framework, demonstrating superior performance across multiple benchmarks compared to methods such as MAPPO and QMIX. Their comprehensive evaluation established HAPPO as a state-of-the-art approach for cooperative multi-agent problems with heterogeneous agents.

Yu et al. (2022) investigated the effectiveness of Proximal Policy Optimization in cooperative multi-agent games, finding that simple adaptations of single-agent PPO could achieve surprisingly strong performance. Their analysis provided insights into why PPO-based methods succeed in multi-agent settings despite the non-stationarity introduced by simultaneously learning agents.

2.2.3. Multi-Agent Approaches for Supply Chain

The intersection of multi-agent reinforcement learning and supply chain management has attracted increasing research attention. Liu et al. (2024) applied HAPPO to multi-echelon inventory management in both serial and network supply chain configurations. Their results demonstrated that multi-agent approaches achieve lower overall costs than single-agent baselines while reducing the bullwhip effect through implicit information sharing during centralized training.

Geevers et al. (2023) studied deep reinforcement learning for multi-echelon inventory optimization across linear, divergent, and general network structures. Their work provided systematic comparisons with heuristic policies and identified conditions under which learning-based approaches offer the greatest advantages.

Kim et al. (2024) proposed a multi-agent reinforcement learning model for inventory transshipment under supply chain disruption, incorporating a fictitious agent to coordinate lateral transfers between retailers. Their approach demonstrated improved resilience compared to traditional policies under pandemic-induced demand volatility.

Khirwar et al. (2023) presented a cooperative multi-agent reinforcement learning system with GPU-parallelized environments for warehouse-store inventory management. Their work addressed scalability challenges and demonstrated the potential for training complex multi-agent policies through distributed computation.

2.2.4. Hybrid and Constrained Approaches

Pure reinforcement learning approaches may propose actions that violate operational constraints or fail to incorporate domain knowledge effectively. Hybrid methods that combine learning with optimization or heuristic refinement offer potential solutions to these limitations.

Bengio et al. (2021) surveyed machine learning approaches for combinatorial optimization, identifying opportunities for integrating neural networks with traditional operations research methods. Their analysis highlighted the complementary strengths of learning-based and optimization-based approaches.

Vanvuchelen et al. (2020) applied Proximal Policy Optimization to the joint replenishment problem, demonstrating that reinforcement learning could handle coordination constraints in multi-product inventory systems. Their work illustrated how action space design and reward shaping could encode domain constraints within the learning framework.

The integration of metaheuristics with reinforcement learning represents another hybrid approach. Genetic algorithms, in particular, offer advantages for lot-sizing subproblems where discrete decisions and complex constraints make gradient-based optimization challenging. The combination of neural network policies for high-level

ordering decisions with genetic algorithm refinement for lot-sizing compliance addresses both adaptability and constraint satisfaction.

2.2.5. Summary and Research Gaps

Table 2.1 summarizes key contributions in the literature and positions the proposed approach relative to existing work.

Table 2-1. Summary of Related Work

No.	Reference	Year	Method	Multi-Agent	Constraint Handling	Key Contribution
1	Clark and Scarf	1960	Analytical	No	Implicit	Echelon base-stock optimality
2	Lee et al.	1997	Analytical	No	N/A	Bullwhip effect characterization
3	Nguyen Van Hop	2006	AGASOR	No	Heuristic	Adaptive GA for scheduling
4	Nguyen Van Hop	2010	DALSA	No	Heuristic	Demand allocation lot-sizing
5	Boute et al.	2022	Survey	Both	Various	DRL roadmap for inventory
6	Gijsbrechts et al.	2022	DRL (PPO)	No	None	Lost sales, dual-sourcing evaluation
7	De Moor et al.	2022	DRL	No	Reward shaping	Perishable inventory
8	Kuba et al.	2022	HAPPO	Yes	None	Monotonic improvement guarantee
9	Yu et al.	2022	MAPPO	Yes	None	PPO effectiveness in MARL
10	Geevers et al.	2023	DRL	No	None	Multi-echelon structures
11	Dehaybe et al.	2024	DRL	No	None	Non-stationary demand
12	Liu et al.	2024	HAPPO	Yes	None	MARL for inventory, bullwhip
13	Kim et al.	2024	MARL	Yes	Transshipment	Disruption resilience
14	Zhong et al.	2024	HARL	Yes	None	Comprehensive HARL framework
	Proposed	**2025**	**HAPPO+ GA**	**Yes**	**GA refinement**	**Multi-objective, lot-sizing**

Research Gaps

The literature review reveals several gaps that motivate the proposed research:

First, while multi-agent reinforcement learning has been applied to inventory management, existing approaches do not explicitly address lot-sizing constraints. Learned policies may propose order quantities that are operationally infeasible, limiting practical applicability.

Second, most reinforcement learning approaches optimize a single objective, typically cost minimization, whereas practitioners require methods that can explore trade-offs between competing objectives such as cost efficiency and service level performance.

Third, the integration of metaheuristic optimization with multi-agent reinforcement learning for inventory management remains unexplored. Such integration could combine the adaptability of learning-based approaches with the constraint satisfaction capabilities of evolutionary algorithms.

This thesis addresses these gaps by developing a hybrid HAPPO and genetic algorithm framework with multi-objective optimization capabilities, as detailed in subsequent chapters.

2.3. Key References & Candidate Solution Methods

2.3.1. HAPPO (Heterogeneous Agent PPO)

Liu et al. (2022) introduced HAPPO to coordinate decentralized inventory decisions via the Centralized Training with Decentralized Execution (CTDE) paradigm. From an operational perspective, HAPPO significantly mitigates the bullwhip effect by employing a centralized critic that enables upstream agents to internalize downstream demand volatility, thereby reducing safety stock requirements across the network. The algorithm utilizes sequential policy updates to ensure stability within the multi-agent environment. However, HAPPO exhibits critical limitations in industrial applications because it treats ordering as an unconstrained process, neglecting minimum order quantities or transport capacities, and its scalarized reward function obscures the necessary trade-offs between cost efficiency and service levels.

2.3.2. Genetic Algorithm for Lot-Sizing

To address the NP-hard nature of capacitated lot-sizing, Nguyen Van Hop (2006; 2010) developed the DALSA and AGASOR frameworks using Genetic Algorithms. These methods distinguish themselves by explicitly encoding hard constraints—such as

truck capacities, supplier-specific lead times, and quantity discounts—directly into the chromosome structure, ensuring that all generated solutions remain logistically feasible. While DALSA effectively optimizes demand allocation among heterogeneous suppliers and AGASOR adapts to dynamic parameters, these metaheuristics remain inherently myopic. They optimize solely within a fixed planning horizon and lack the reinforcement learning capability to retain historical experiences or adapt to long-term distributional shifts in demand.

2.3.3. Hybrid Planner Approach

The hybrid planner architecture synergizes the adaptive foresight of Reinforcement Learning with the constraint enforcement of Genetic Algorithms. In this framework, the neural network policy generates candidate order quantities based on observed global states, capturing long-term stochastic dynamics that purely heuristic methods miss. These candidates subsequently serve as "warm starts" for the Genetic Algorithm, which refines the quantities to strictly satisfy operational constraints such as packaging units and vehicle limits. This integration ensures that procurement decisions are both strategically adaptive to evolving demand patterns and operationally compliant with rigid commercial requirements.

2.3.4. Multi-Objective Optimization Framework

Inventory management involves inherent trade-offs between cost efficiency and service level performance. Lower inventory investment reduces holding costs but increases stockout risk, while higher safety stock improves service at the expense of capital tied up in inventory.

Rather than committing to a single point on this trade-off frontier, the proposed framework supports multi-objective optimization through configurable objective weights. By varying the relative importance assigned to cost minimization versus service level maximization, the framework can generate a set of Pareto-efficient policies representing different operating points.

This capability enables practitioners to explore the trade-off frontier and select policies aligned with strategic priorities. For instance, a retailer focused on customer experience might prefer a policy achieving high service levels at moderate cost, while a

cost-driven organization might accept lower service levels in exchange for reduced inventory investment.

CHAPTER 3: METHODOLOGY

3.1. Approaches Comparison and Selection

3.1.1. Qualitative Comparison

Table 3.1 presents a qualitative comparison of the three methodological approaches across four evaluation dimensions: adaptability to demand uncertainty, constraint handling capability, computational efficiency during inference, and coordination mechanism for multi-agent settings.

Table 3-1. Qualitative comparison of methodological approaches

Paradigm	Adaptability to Stochasticity	Hard Constraint Compliance	Inference Speed	Network Coordination
Classical (EOQ/POQ/Min-Max)	Low: Assumes stationary (μ, σ^2)	Medium: Constraints handled via rounding	Very High: O(1)	None: Echelons operate independently
Meta-Heuristics (GA/DALSA)	Medium: Scenario-based evaluation	High: Feasibility guaranteed through encoding	Low: O(G×P×H)	Low: No coordinated learning
End-to-End RL (HAPPO)	High: Adapts online to shifts	Low: Continuous outputs violate MOQ/packaging	High: O(d) forward pass	High: Centralized critic coordinates
Proposed Hybrid	High: RL captures dynamics	High: GA guarantees feasibility	High: O(d) + O(G×P×H)	High: RL coordinates, GA executes

Note: G = generations, P = population size, H = planning horizon, d = observation dimension.

The analysis reveals a fundamental tension in existing approaches. Classical methods offer computational efficiency and constraint satisfaction but fail under non-stationary demand conditions prevalent in modern supply chains. Pure reinforcement learning achieves superior adaptability and coordination but struggles with hard operational constraints. Meta-heuristics handle constraints effectively but lack the temporal learning capability to anticipate demand evolution. Only the hierarchical hybrid paradigm achieves high performance across all dimensions simultaneously, motivating its selection as the foundation for the proposed framework.

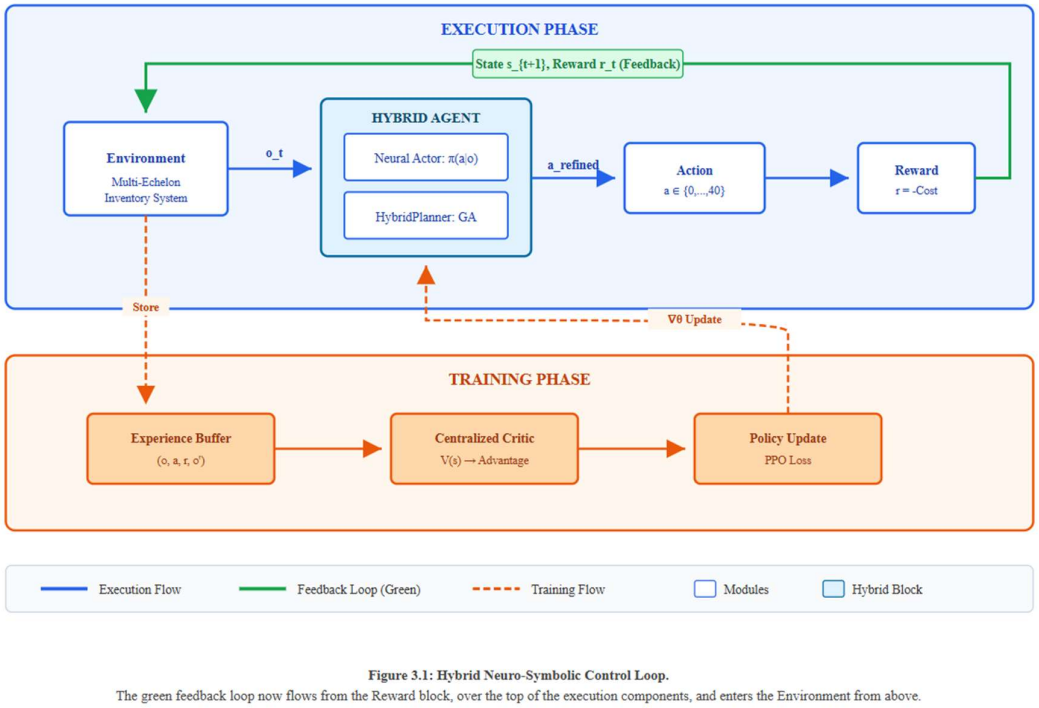


Figure 3.1: Hybrid Neuro-Symbolic Control Loop.
The green feedback loop now flows from the Reward block, over the top of the execution components, and enters the Environment from above.

Figure 3-1. Hybrid Neuro-Symbolic Control Loop

3.1.2. Quantitative and Logical Justification

Critique of Classical Methods. EOQ and (s,S) policies assume demand stationarity: $D_t \sim F(\mu, \sigma^2)$ for all t. Contemporary supply chains violate this assumption through bullwhip amplification, where $\boldsymbol{Var(Order_i) > Var(Order_{i-1})}$ as orders propagate upstream. Static parameters become obsolete within weeks of calibration. Furthermore, these methods optimize single-objective cost functions and cannot dynamically navigate the service level versus cost trade-off.

Critique of Pure Heuristics. Genetic algorithms optimize over finite horizons without terminal value estimation. The objective $min \sum_{t=1}^{H}[h \cdot I_t^+ + b \cdot I_t^- + K \cdot \delta(Q_t)]$ creates systematic myopia, as solutions may deplete safety stocks near the horizon boundary without considering periods $H+1, H+2, \ldots$. Unlike RL, which estimates $V(s) = \mathbb{E}[\sum_{t=1}^{\infty} \gamma^t r_t]$, meta-heuristics lack temporal difference learning to anticipate future demand spikes. Each optimization cycle starts fresh without exploiting patterns from previous episodes.

Critique of Pure RL. Direct mapping to order quantities creates action space explosion: $|A| = K^N = 68{,}921$ joint actions for a three-echelon system with K=41K=41 K = 41 options. The fixed ordering cost $K \cdot \delta(Q_t)$ introduces step-function

discontinuities that impede gradient-based learning. Neural networks produce continuous distributions requiring discretization, and post-hoc projection to satisfy MOQ or packaging constraints corrupts gradient estimates.

3.1.3. Selection Rationale

The hierarchical approach is selected to decouple the problem into two tractable sub-problems:

Strategic Director (RL Agent): Learns policy $\pi_\theta(a|s)$ for handling uncertainty and network coordination. The RL component exploits temporal difference learning to anticipate demand evolution and uses centralized training to coordinate decisions across echelons, thereby mitigating bullwhip amplification.

Tactical Executor (Heuristic Solver): Translates RL triggers into feasible order quantities through constrained optimization:

$$Q_t \in \{0\} \cup \{Q : Q \geq MOQ, Q \bmod LSM = 0, Q \leq CAP\}$$

This separation ensures the system is Adaptive (via RL temporal learning), Coordinated (via centralized critic), and Compliant (via constrained optimization). Section 3.2 details the architectural instantiation of this paradigm.

3.2. Proposed System Design or Proposed Solution Approach

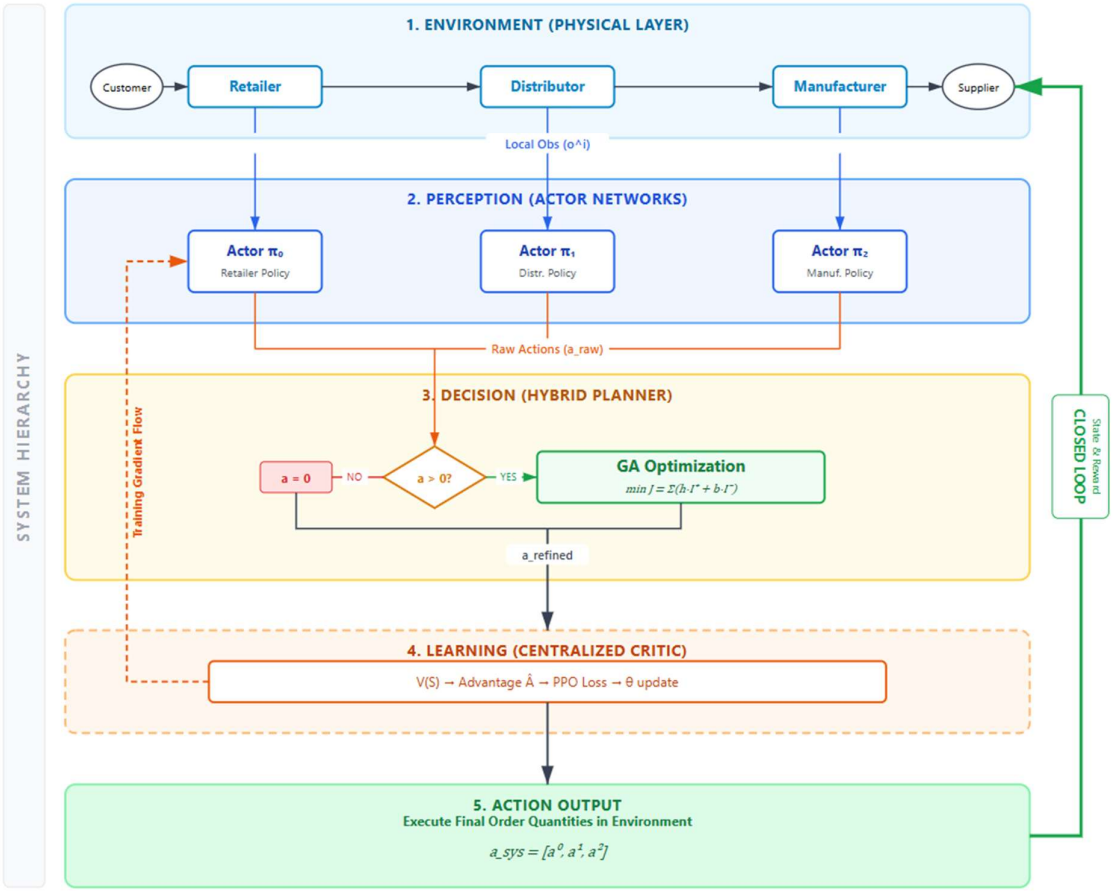


Figure 3-2. System Architecture Overview - Hierarchical Data Flow

3.2.1. System Architecture Overview

The proposed system integrates multi-agent deep reinforcement learning with classical lot-sizing heuristics through a hierarchical control architecture. This hybrid approach addresses the fundamental challenge of balancing exploration-exploitation in high-dimensional action spaces while maintaining interpretable decision-making aligned with established inventory management principles.

The architecture follows a Centralized Training with Decentralized Execution (CTDE) paradigm, where a centralized critic network processes global state information during training while individual agents execute decisions based solely on local observations during deployment. This design enables sophisticated coordination learning without requiring communication infrastructure during operation.

The system operates under asymmetric penalty structures that reflect real-world supply chain economics. Specifically, the backlog cost vector **b = [5, 3, 2]** assigns progressively higher penalties to downstream echelons, where $b_0 = 5$ for the retailer, b_1

= 3 for the distributor, and $b_2 = 2$ for the manufacturer. This configuration captures the economic reality that stockouts at customer-facing nodes incur greater consequences through lost sales, customer dissatisfaction, and potential long-term demand erosion. The holding cost remains uniform at **h = [1, 1, 1]** across all echelons, creating a cost ratio that incentivizes the system to maintain higher service levels at downstream positions while accepting moderate inventory accumulation upstream.

The control loop operates at each time step t through a sequential process. First, each agent i observes its local state including inventory position, backlog level, downstream demand signal, and pipeline orders. Second, the decentralized actor network generates a raw action representing the intended order quantity. Third, the HybridPlanner conditionally refines this action using genetic algorithm optimization when the raw action exceeds zero. Finally, the refined action is executed in the environment, triggering state transitions and cost accumulation.

3.2.2. Multi-Objective Reward Formulation

Inventory management inherently involves trade-offs between cost efficiency and service level performance. Lower inventory investment reduces holding costs but increases stockout risk, while higher safety stock improves customer service at the expense of capital tied up in inventory. Rather than committing to a single point on this trade-off frontier, the proposed framework employs a weighted scalarization approach that enables systematic exploration of Pareto-efficient solutions.

The multi-objective reward function transforms individual agent rewards by combining cost-based and service-based components through configurable weights. For each agent *i*, the multi-objective reward is computed as:

$$R_i^{MO} = \alpha \cdot R_i^{cost} + \beta \cdot FR_i \cdot \gamma_{scale}$$

where R_i^{cost} denotes the cost-based reward (negative of incurred costs) for agent *i*, FR_i represents the fill rate (ratio of fulfilled demand to total demand) for agent *i*, $\alpha \in [0,1]$ is the weight assigned to cost minimization, $\beta \in [0,1]$ is the weight assigned to service level maximization with the constraint $\alpha + \beta = 1$, and $\gamma_{scale} = 10.0$ is a scaling factor that normalizes the service component to be comparable in magnitude to the cost component.

The fill rate FR_i is computed as the proportion of demand satisfied from available inventory:

$$FR_i = \text{Fulfilled Demand}_i \,/\, \text{Total Demand}_i$$

This formulation enables the framework to generate policies spanning the entire Pareto frontier. When $\alpha = 1$ and $\beta = 0$, the optimization focuses exclusively on cost minimization, potentially accepting lower service levels. Conversely, when $\alpha = 0$ and $\beta = 1$, the optimization prioritizes service level maximization regardless of cost implications. Intermediate weight configurations produce policies that balance both objectives according to practitioner preferences.

Five weight configurations are investigated to characterize the Pareto frontier comprehensively, as presented in Table 3.2.

Table 3-2. Pareto Weight Configurations for Multi-Objective Optimization

Configuration	α (Cost Weight)	β (Service Weight)
Cost-focused	1.0	0.0
Cost-leaning	0.7	0.3
Balanced	0.5	0.5
Service-leaning	0.3	0.7
Service-focused	0.0	1.0

Each configuration is trained independently with three random seeds to ensure statistical reliability. The resulting policies collectively trace the Pareto frontier, enabling decision-makers to select operating points aligned with their strategic priorities without requiring computationally intensive multi-objective evolutionary algorithms.

3.2.3. Observation Space Design

The multi-echelon inventory management problem is formulated as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), a mathematical framework that captures the essential characteristics of cooperative decision-making under uncertainty. This formulation accommodates the information asymmetry inherent in supply chain operations, where each echelon possesses only local visibility into system state.

Formal Definition. The system is characterized by the tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where

- $\mathcal{N} = \{0,1,\dots,N-1\}$ denotes the set of $\mathcal{N}$ agents corresponding to supply chain echelons,
- $\mathcal{S}$ represents the joint state space,
- $\mathcal{A}$ constitutes the joint action space,
- $\mathcal{P}: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0,1]$ defines the state transition probability function,
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^N$ specifies the reward function, and
- $\gamma \in [0,1)$ represents the discount factor.

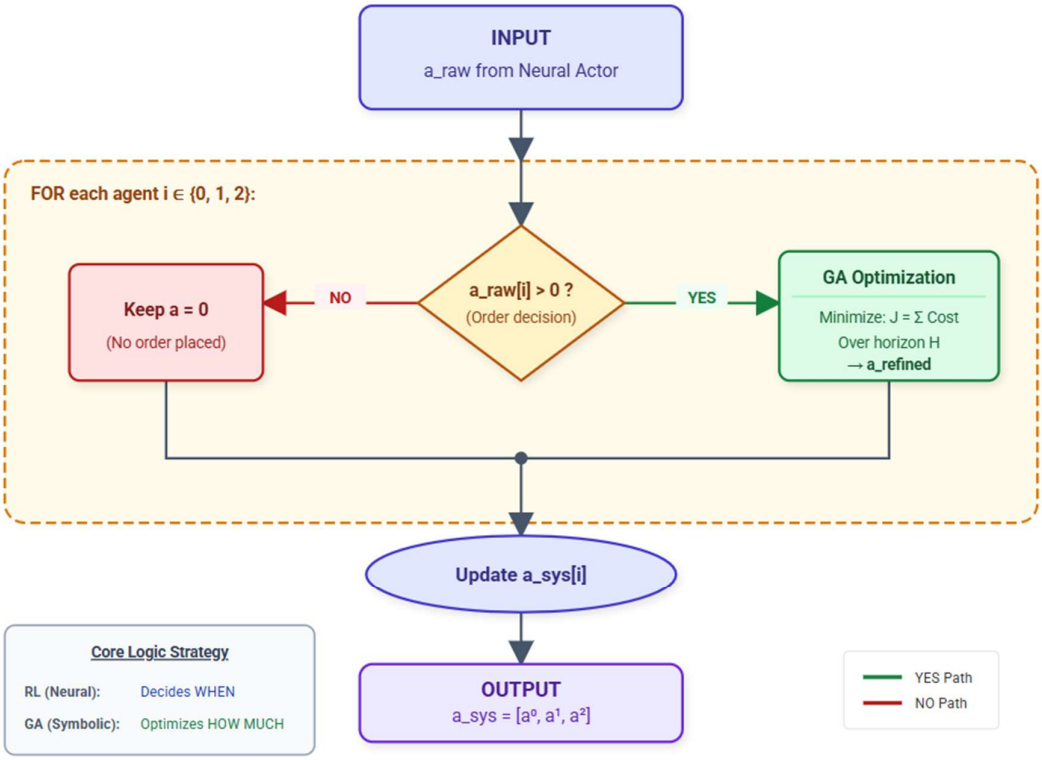
Partial Observability. Each agent i observes a local state vector s_t^i rather than the complete system state. This observation encompasses the agent's inventory position, backlog status, downstream demand signal, and pipeline orders awaiting arrival. The downstream demand signal exhibits positional heterogeneity: terminal echelons observe exogenous customer demand, while upstream echelons observe order quantities from immediate downstream neighbors. This information structure captures the demand signal distortion that underlies bullwhip amplification.

Reward Structure. The reward function encapsulates the multi-objective nature of inventory management, translating operational costs into learning signals. Each agent receives an individual reward computed as the negative of incurred costs, comprising holding expenses for on-hand inventory, backlog penalties for unfulfilled demand, and fixed costs for order placement. The asymmetric penalty structure assigns higher backlog costs to downstream echelons, creating differentiated incentives aligned with customer service priorities.

Coordination Mechanism. To promote cooperative behavior despite individual reward signals, the framework incorporates reward smoothing that blends individual performance with team average outcomes. This mechanism discourages policies that externalize costs onto neighboring echelons, encouraging emergent coordination without requiring explicit communication protocols or centralized decision-making during execution.

3.2.4. Hybrid Refinement Mechanism

The hybrid refinement mechanism serves as the critical interface between learned policies and operational execution. This component addresses a fundamental limitation of pure neural network approaches: the difficulty of satisfying hard constraints while maintaining differentiability for gradient-based optimization.



Detailed decision flow of the HybridPlanner. The Neural Actor first determines the binary decision (To order or not?). If positive, the Genetic Algorithm (GA) refines the precise order quantity to minimize costs over the horizon H.

Figure 3-3. HybridPlanner - Decision and Refinement Logic

Conceptual Design. The refinement mechanism operates on a conditional logic that preserves the semantic content of policy decisions while ensuring operational feasibility. When an agent signals zero ordering intent, this decision propagates unchanged, allowing the learned policy to express "no-order" recommendations when inventory positions are adequate. When an agent signals positive ordering intent, the mechanism invokes a local optimization procedure that determines the specific quantity minimizing expected costs over a finite planning horizon.

Rationale for Conditional Activation. The binary structure of the refinement trigger serves multiple purposes. First, it reduces computational overhead by invoking optimization only when necessary. Second, it preserves the policy's ability to learn temporal patterns in ordering behavior, distinguishing between periods requiring replenishment and those where existing inventory suffices. Third, it creates a natural separation between strategic decisions (whether to order) and tactical decisions (how much to order), aligning with hierarchical planning principles established in the operations research literature.

Local Optimization Objective. The genetic algorithm component minimizes a local cost function over a finite planning horizon, considering holding costs for anticipated inventory accumulation, backlog penalties for projected stockouts, and fixed costs for order placement. The asymmetric backlog penalties ensure that optimization prioritizes stockout avoidance more aggressively for downstream agents, aligning tactical lot-sizing decisions with strategic service level objectives.

Constraint Satisfaction. The genetic algorithm representation inherently satisfies operational constraints through its encoding structure. Minimum order quantities, packaging increments, and capacity bounds are embedded directly in the solution representation, ensuring that all candidate solutions remain feasible without requiring penalty-based constraint handling that could distort the cost landscape.

Training Stability Mechanisms. To ensure robust convergence and prevent overfitting, the training protocol incorporates early stopping with learning rate adaptation. These mechanisms monitor policy performance during training and terminate optimization when further improvement becomes unlikely.

The early stopping procedure operates as follows. During an initial warmup phase spanning the first 10 evaluation cycles, no stopping decisions are made, allowing the policy sufficient exploration time. Subsequently, training continues until the evaluation metric fails to improve by at least 1% for 15 consecutive evaluation cycles, at which point training terminates and the best-performing checkpoint is retained.

Learning rate reduction complements early stopping by adapting the optimization step size when progress stalls. When the evaluation metric fails to improve for 5 consecutive cycles, the learning rates for both actor and critic networks are reduced by

a factor of 0.5. This reduction enables finer-grained policy refinement when the optimization landscape flattens, potentially escaping local optima that would otherwise trigger early termination.

Table 3-3. Early Stopping and Learning Rate Adaptation Configuration

Parameter	Value	Description
Warmup evaluations	10	Initial phase without early stopping
Patience	15	Evaluations without improvement before stopping
Minimum improvement	1%	Threshold for counting as improvement
LR reduction factor	0.5	Multiplier applied when plateau detected
LR reduction patience	5	Evaluations before reducing learning rate

3.2.5. Learning Algorithm Framework

The learning algorithm integrates policy gradient optimization with value function estimation through the Proximal Policy Optimization (PPO) framework, adapted for multi-agent settings through the Heterogeneous Agent PPO (HAPPO) formulation.

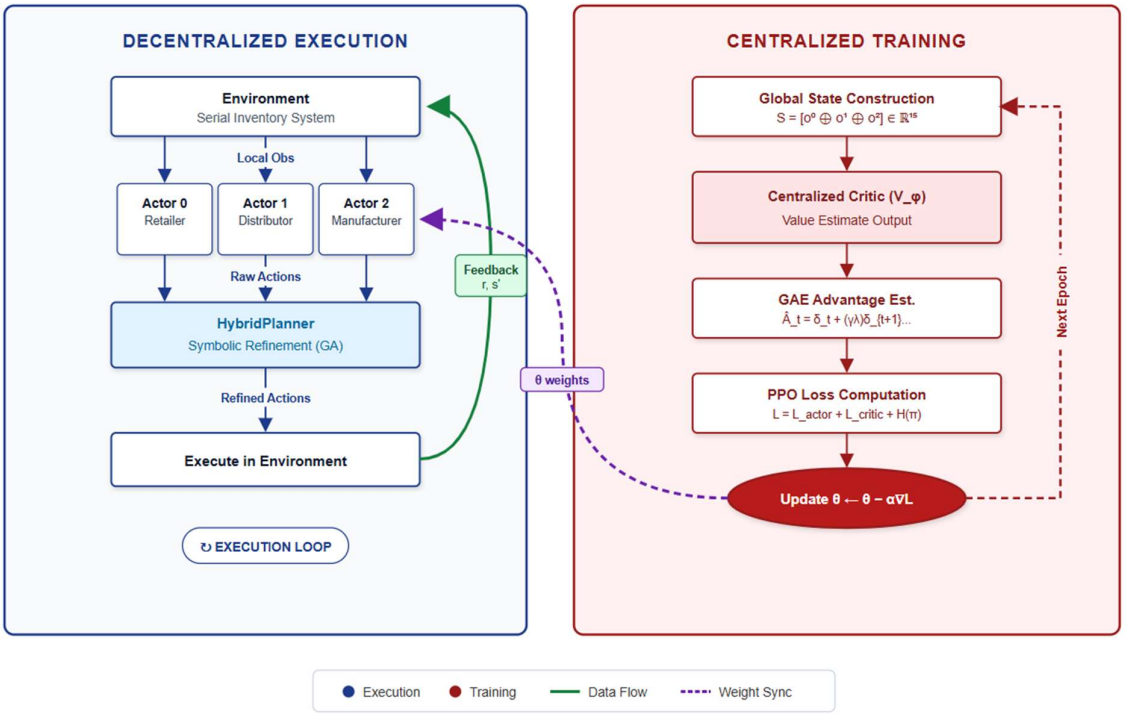


Figure 3-4. CTDE Framework for Multi-Echelon Inventory Optimization

Decentralized Policy Learning. Each agent maintains an independent policy network that maps local observations to action distributions. The heterogeneous formulation permits agents to develop specialized behaviors appropriate to their supply chain position, rather than forcing parameter sharing that would constrain

representational capacity. Policy updates follow the proximal policy optimization objective, which constrains the magnitude of policy changes to ensure stable learning progression.

Centralized Value Estimation. A shared critic network estimates the global state value by processing concatenated observations from all agents. This centralized architecture enables credit assignment that accounts for the delayed and distributed consequences of ordering decisions. The critic provides baseline estimates for advantage computation, reducing variance in policy gradient estimation while enabling agents to internalize the system-wide impact of their actions.

Advantage Estimation. The framework employs Generalized Advantage Estimation (GAE) to compute policy gradients, balancing bias and variance through a tunable parameter. This technique enables efficient use of trajectory data while maintaining the stability benefits of baseline subtraction. The temporal difference errors underlying GAE capture the incremental value of actions, enabling agents to learn from the immediate and delayed consequences of ordering decisions.

Trust Region Enforcement. The PPO clipping mechanism constrains policy updates to a trust region, preventing catastrophic policy degradation that can occur with large gradient steps. This stability guarantee is particularly important in multi-agent settings, where simultaneous policy updates create non-stationarity that can destabilize learning. The clipped objective ensures monotonic improvement guarantees hold approximately, enabling reliable convergence to effective coordination strategies.

Entropy Regularization. The learning objective incorporates entropy bonuses that encourage policy exploration during early training phases. This regularization prevents premature convergence to deterministic policies that may represent local optima, enabling discovery of superior coordination strategies through continued exploration of the action space.

CHAPTER 4: SOLUTION DEVELOPMENT

4.1. Model / Prototype Solution

The overall structure of the training system is depicted in **Figure 4.1**. This diagram highlights the closed-loop interaction between the decision-making block (Agent System) and the simulation environment. A key feature of this architecture is the inclusion of the **Hybrid Planner** as an intermediate layer for action refinement, combined with the PPO parameter update mechanism and periodic evaluation process, ensuring stable model convergence regarding target metrics such as Fill Rate and Total Cost

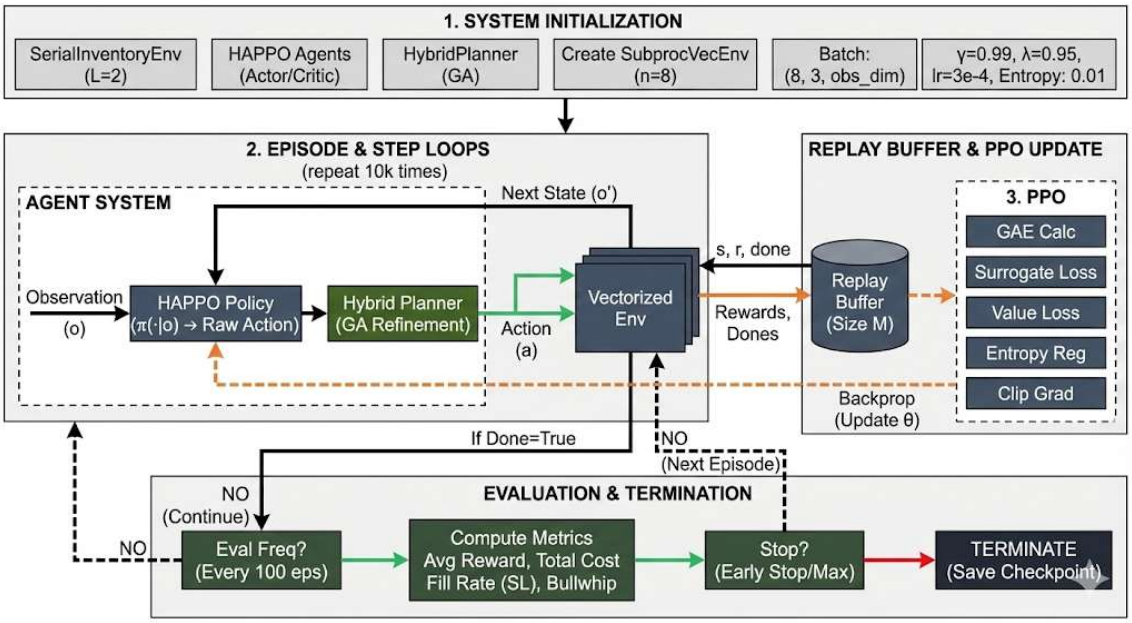


Figure 4-1. The proposed Hybrid-HAPPO training framework.

4.1.1. Mathematical Formulation

The multi-echelon inventory management problem is formulated as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP), wherein multiple autonomous agents coordinate replenishment decisions under demand uncertainty. This formulation captures the inherent challenges of information asymmetry and sequential decision-making across supply chain echelons.

Markov Decision Process Tuple

The system is formally defined by the tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{N} = \{0,1, \dots, N-1\}$ denotes the set of $\mathcal{N}$ agents corresponding to supply chain echelons, $\mathcal{S}$ represents the joint state space, $\mathcal{A}$ constitutes the joint action space, $\mathcal{P}: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0,1]$ defines the state transition probability function, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^N$ specifies the reward function, and $\gamma \in [0,1)$ represents the discount factor.

State Space

Each agent $i \in \mathcal{N}$ observes a local state vector s_t^i at discrete time step t. The observation comprises four components that collectively characterize the agent's operational context:

$$s_t^i = \left[I_t^i, B_t^i, D_t^i, P_{t,1}^i, P_{t,2}^i, \dots, P_{t,L}^i\right] \in \mathbb{R}^{L+3}$$

Where:

$I_t^i \in \mathbb{R}_{\geq 0}$ denotes the on-hand inventory level at echelon i

$B_t^i \in \mathbb{R}_{\geq 0}$ represents the backlog quantity (unfulfilled demand)

$D_t^i \in \mathbb{R}_{\geq 0}$ corresponds to downstream information: exogenous customer demand for retailers (i=0) or the order quantity from the immediate downstream echelon (i>0)

$P_{t,1}^i \in \mathbb{R}_{\geq 0}$ for $k \in \{1, \dots, L\}$ indicates pipeline orders scheduled to arrive in k periods

The observation dimensionality is thus $|s_t^i| = L + 3$. With lead time L=2, each agent observes a 5-dimensional state vector. The global state is constructed as the concatenation of all local observations:

$$s_t = [s_t^0, s_t^1, \dots, s_t^{N-1}] \in \mathbb{R}^{N(L+3)}$$

Action Space

The action space is discretized to facilitate tractable policy optimization. Each agent *i* selects an action a_i^t representing the order quantity to be placed with its upstream supplier:

$$a_i^t \in \mathcal{A}^i = \{0,1,2, \dots, A_{max} - 1\}$$

Where $A_{max} = 41$ defines the action space cardinality. This discretization provides sufficient granularity for practical replenishment decisions while maintaining computational feasibility. The joint action vector is defined as $a_t = (a_t^0, a_t^1, \dots, a_t^{N-1})$.

Reward Function

The reward function encapsulates the multi-objective optimization criteria inherent to inventory management. Each agent receives an individual reward signal computed as the negative of incurred costs:

$$R_t^i = -\left(h_i \cdot I_t^i + b_i \cdot B_t^i + f \cdot 1_{[a_t^i > 0]}\right)$$

The cost parameters are configured with asymmetric backlog penalties to reflect the economic reality that stockouts at downstream echelons (closer to end customers) incur greater penalties:

Table 4-1. Cost parameter across 3 echelons

Parameter	**Retailer (*i*=0)**	**Distributor (*i*=1)**	**Manufacturer (*i*=2)**
Holding cost h_i	1.0	1.0	1.0
Backlog cost b_i	5.0	3.0	2.0
Fixed cost f	1.0	1.0	1.0

Reward Smoothing for Coordination

To promote inter-agent coordination during training, the individual rewards are smoothed using a convex combination with the team average:

$$\tilde{R}_t^i = \alpha \cdot R_t^i + (1 - \alpha) \cdot \bar{R}_t$$

Where $\bar{R}_t = \frac{1}{N}\sum_{j=0}^{N-1} R_t^j$ represents the mean reward across all agents and $\alpha = 0.5$ balances individual and collective performance. This mechanism mitigates reward disparity and encourages cooperative behavior.

Multi-Objective Reward Implementation

The multi-objective reward function extends the base cost-based reward with a service level component. The implementation computes rewards per agent as follows:

$$R_i^{MO} = \alpha \cdot R_i^{cost} + \beta \cdot FR_i \cdot \gamma_{scale}$$

The implementation details for each component are specified below.

Table 4-2. Multi-Objective Reward Component Specifications

Component	**Implementation**	**Computation**
R_i^{cost}	raw_rewards[i]	$-(h_i \cdot I_t^i + b_i \cdot B_t^i + f \cdot \mathbb{1}[a_t^i > 0])$
FR_i	fill_rates[i]	$\min(\text{Available}_i, \text{Demand}_i) / \text{Demand}_i$

α	cost_weight	Weight for cost objective, α ∈ [0,1]
β	service_weight	Weight for service objective, β = 1 − α
γ_{scale}	service_bonus_scale	Scaling factor = 10.0 (normalizes magnitude)

Fill Rate Computation. The environment tracks demand and fulfillment history per agent. At each time step, the fill rate is computed as the ratio of fulfilled quantity to effective demand:

$$fulfilled_i = \min(inventory_i + received_i, effective_demand_i)$$

$$FR_i = fulfilled_i / effective_demand_i$$

where $effective_demand_i = current_demand_i + backlog_i$ accounts for unfulfilled demand from previous periods.

4.1.2. Neural Network Specifications

The policy architecture employs the Centralized Training with Decentralized Execution (CTDE) paradigm. This design enables agents to leverage global information during training while maintaining independent execution capabilities during deployment.

Actor Network Architecture

Each agent i maintains an independent actor network $\pi_{\theta_i} : \mathbb{R}^{L+3} \rightarrow \mathbb{R}^{|\mathcal{A}^i|}$ that maps local observations to action logits. The architecture comprises a three-layer Multi-Layer Perceptron (MLP):

$$z^i = W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 \cdot s^i + b_1) + b_2) + b_3$$

Where $\sigma(\cdot)$ denotes the Rectified Linear Unit (ReLU) activation function, $\{W_k, b_k\}_{k=1}^{3}$ constitute the learnable parameters, and $z^i \in \mathbb{R}^{A_{max}}$ represents the output logits. Action sampling is performed by constructing a Categorical distribution from the logits:

$$a^i \sim Categorical(logits = z^i)$$

The network topology is specified as:

Table 4-3. Actor Network Architecture details

Layer	**Input Dimension**	**Output Dimension**	**Activation**
Input	$L + 3 = 5$	128	ReLU
Hidden	128	128	ReLU

Output	128	$A_{max} = 41$	None (logits)

4.1.3. Algorithm Parameters

The Heterogeneous-Agent Proximal Policy Optimization (HAPPO) algorithm governs the policy learning process. The hyperparameter configuration balances exploration and exploitation while ensuring stable convergence:

Table 4-4. HAPPO Training Hyperparameters

Parameter	Symbol	Value	Justification
Actor learning rate	η_θ	1×10^{-4}	Conservative rate for stable policy updates
Critic learning rate	η_φ	1×10^{-4}	Matched with actor for balanced learning
Discount factor	γ	0.99	High value emphasizes long-term rewards
GAE lambda	λ	0.95	Bias-variance trade-off in advantage estimation
PPO clip range	ε	0.2	Standard clipping for trust region enforcement
Value coefficient	c_v	0.5	Relative weight of value loss
Entropy coefficient	c_e	0.05	Encourages exploration during early training
Training episodes	K	**5,000**	Sufficient iterations for convergence
Episode horizon	T	100	Standard planning horizon
Rollout threads	M	8	Parallel environments for sample efficiency
Evaluation frequency	—	**50 episodes**	Periodic performance assessment
Early stopping patience	—	**15 evaluations**	Prevents overfitting

Pareto Multi-Objective Training Configuration

The framework trains separate models for each point on the Pareto frontier using the weight configurations specified below. Each configuration is trained with three random seeds to ensure statistical reliability.

Table 4-5. Pareto Weight Configurations

Configuration Name	α (cost_weight)	β (service_weight)	Seeds
cost_focused	1.0	0.0	[1, 2, 3]
cost_leaning	0.7	0.3	[1, 2, 3]
balanced	0.5	0.5	[1, 2, 3]
service_leaning	0.3	0.7	[1, 2, 3]

service_focused	0.0	1.0	[1, 2, 3]

This configuration yields 15 trained models per topology (5 configurations × 3 seeds), enabling statistical comparison across the Pareto frontier.

4.1.4. GA Lot-Sizing Parameters

The Genetic Algorithm (GA) component refines replenishment decisions by optimizing order quantities over a finite planning horizon. This heuristic complements the learned policy by incorporating domain-specific lot-sizing constraints.

Table 4-6. Genetic Algorithm Configuration

Parameter	Value	Description
Population size	15	Number of candidate solutions per generation
Generations	20	Maximum evolutionary iterations
Mutation rate	0.1	Probability of random gene perturbation
Planning horizon	5	Periods considered in optimization
Selection method	Roulette wheel	Fitness-proportional parent selection
Crossover operator	Single-point	Standard genetic recombination
Elitism	Enabled	Best individual preserved across generations

Fitness Function

The GA evaluates candidate order plans $\boldsymbol{p} = [p_0, p_1, \dots, p_{H-1}]$ using a cost-based fitness function:

$$F(\boldsymbol{p}) = -\sum_{t=0}^{H-1} \left(h \cdot I_t(\boldsymbol{p}) + b \cdot B_t(\boldsymbol{p}) + f \cdot 1_{[p_t > 0]} \right)$$

where $I_t(\boldsymbol{p})$ and $B_t(\boldsymbol{p})$ denote the inventory and backlog levels resulting from executing plan **p** given the demand forecast. The algorithm returns the first-period order quantity $\boldsymbol{p}_0^*$ from the optimal plan as the refined action.

4.2. Solution Development

4.2.1. Solution Approaches Overview

Three solution approaches are investigated in this thesis:

Model	Approach	Description

Model 1	Hierarchical Rule Selection	Meta-learner selects among FOQ, POQ, Silver-Meal rules
Model 2	Pure MARL	HAPPO framework
Model 3	Hybrid MARL + GA	Proposed integration of HAPPO with GA refinement

Models 1 and 2 serve as baselines for comparative evaluation in Chapter 5. This section focuses on the implementation details of Model 3.

4.2.2. Neural Network Architecture

The proposed framework employs the Centralized Training with Decentralized Execution (CTDE) paradigm with heterogeneous actor networks and a shared centralized critic.

Actor Network

Each agent i maintains an independent policy network π_{θ_i} that maps local observations to action distributions:

$$\pi_{\theta_i}\left(a \middle| O_t^i\right) = Categorial(softmax(f_{\theta_i}(O_t^i)))$$

Where softmax converts logits to probabilities:

$$softmax(\boldsymbol{z})_j = \frac{\exp(z_j)}{\sum_{k=0}^{K-1} \exp(z_k)}$$

The actor architecture consists of a two-layer MLP with ReLU activations:

$$f_{\theta_i}: \mathbb{R}^5 \xrightarrow{Linear} \mathbb{R}^{128} \xrightarrow{ReLU} \mathbb{R}^{128} \xrightarrow{ReLU} \mathbb{R}^{41}$$

Critic Network

The centralized critic $V_\phi(S_t)$ estimates the global state value by processing concatenated observations from all agents:

$$S_t = [O_t^1 \oplus O_t^2 \oplus .. \oplus O_t^n] \in \mathbb{R}^{n \times d}$$

For a 3-agent serial supply chain with 5-dimensional observations:

- Global state dimension: state_dim = $n \times d = 3 \times 5 = 15$

The critic network employs a larger architecture:

$$V_\phi: \mathbb{R}^{15} \xrightarrow{Linear} \mathbb{R}^{256} \xrightarrow{ReLU} \mathbb{R}^{256} \xrightarrow{ReLU} \mathbb{R}^1$$

Table 4-7. Network Specifications

Component	Input	Hidden Layers	Output	Activation	Parameters
Actror (x3)	5	[128, 128]	41	ReLU, softmax	~22K each
Critic (x1)	15	[256, 256]	1	ReLU	~70K

4.2.3. Training Algorithm

The training procedure follows the Heterogeneous Agent Proximal Policy Optimization (HAPPO) framework with Generalized Advantage Estimation.

Generalized Advantage Estimation (GAE)

Advantages for policy gradient updates are computed using GAE:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

where the TD error is:

$$\delta_t = r_t + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$$

The global reward aggregates individual rewards:

$$r_t = \sum_{i=1}^{n} r_t^i = -\sum_{i=1}^{n} (h_i \cdot I_t^+ + b_i \cdot I_t^- + 1(Q_t > 0) \cdot C_f)$$

PPO Clipped Objective

Each actor is updated using the PPO clipped surrogate objective:

$$L^{CLIP}(\theta_i) = \mathbb{E}_t\left[\min(\rho_t(\theta_i)\hat{A}_t\,, clip(\rho_t(\theta_i), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right]$$

where $\rho_t(\theta_i) = \frac{\pi_{\theta_i}(a_t|O_t^i)}{\pi_{\theta_i^{old}}(a_t|O_t^i)}$ is the probability ratio.

Loss Functions

Actor loss with entropy regularization:

$$\mathcal{L}_{actor}^i = -L^{CLIP}(\theta_i) \; - \; \beta \cdot H\left[\pi_{\theta_i}\right]$$

where $\beta = 0.05$ is the entropy coefficient encouraging exploration.

Critic loss (MSE): $\mathcal{L}_{critic} = \mathbb{E}_t\left[(V_\phi(S_t) - \hat{R}_t)^2\right]$

Where $\hat{R}_t = \hat{A}_t + V_\phi(S_t)$

Training Procedure

Algorithm 1: HAPPO Training Loop

Input: Environment env, episodes E, horizon T

```
Output: Trained actors {π_θi}, critic V_φ

Initialize actors {π_θi}, critic V_φ, buffer B
for episode = 1 to E do
    obs ← env.reset()
    for t = 1 to T do
        for each agent i do
            a_t^i ~ π_θi(·|o_t^i)
            log_prob_t^i ← log π_θi(a_t^i|o_t^i)
        end for
        actions ← HybridPlanner.refine([a_t^0, a_t^1, a_t^2])
        obs', rewards, done ← env.step(actions)
        B.store(obs, actions, log_probs, rewards, obs', done)
        obs ← obs'
    end for

    // Compute advantages using GAE
    advantages ← compute_GAE(B, V_φ, γ, λ)
    returns ← advantages + V_φ(states)

    // Update critic
    L_critic ← MSE(V_φ(states), returns)
    φ ← φ - η_c · ∇_φ L_critic

    // Update each actor
    for each agent i do
        L_actor^i ← -L^CLIP(θ_i) - β · H[π_θi]
        θ_i ← θ_i - η_a · ∇_θi L_actor^i
    end for

    B.clear()
```

```
30: end for
```

Early Stopping Implementation

The training protocol employs an EarlyStoppingTracker class that monitors evaluation metrics and terminates training when convergence criteria are satisfied. The tracker implements three stopping conditions:

Condition 1 (Primary): No improvement in the primary metric (combined score) for *patience* consecutive evaluations after the warmup phase.

Condition 2 (Trend Detection): Sustained deterioration detected over a sliding window of recent evaluations.

Condition 3 (LR Reduction): When plateau is detected but stopping criteria not yet met, learning rate is reduced to enable finer optimization.

Table 4-8. Early Stopping Configuration Parameters

Parameter	Symbol	Value	Description
warmup_evaluations	—	10	No stopping during warmup
patience	—	15	Evals without improvement
min_delta	δ	0.01	1% relative improvement
trend_window	—	5	Window for trend detection
reduce_lr_on_plateau	—	True	Enable LR reduction
lr_reduction_factor	—	0.5	LR multiplier on plateau
lr_reduction_patience	—	5	Evals before LR reduction
min_lr	—	10^{-6}	Minimum learning rate

Improvement Check. The tracker uses relative improvement to determine progress:

$$\text{improved} = (\text{score} - \text{best_score}) / |\text{best_score}| > \delta$$

This relative criterion ensures consistent behavior across different weight configurations, where absolute reward scales vary significantly between cost-focused and service-focused objectives.

4.2.4. Hybrid Planner Module

The Hybrid Planner serves as the interface between learned policies and operational execution, ensuring constraint satisfaction through conditional GA refinement.

Conditional Activation Logic

The HybridPlanner serves as the critical interface between learned policies and classical optimization. Given a raw RL action a_{rav}, the planner applies conditional refinement:

$$a_{sys} = \begin{cases} a_{rav}(unchanged) & if\ a_{rav} \leq 0 \\ GA(I_t, forecast, \Theta\text{cost}) & otherwise \end{cases}$$

This design serves multiple purposes:

- Reduces computational overhead (GA only when necessary)
- Preserves policy's ability to express "no-order" decisions
- Separates strategic decisions (whether to order) from tactical decisions (how much)

Genetic Algorithm Formulation

Chromosome Representation: An individual is a vector of order quantities over planning horizon *H:*

$$\mathbf{x} = [x_0, x_1, \dots, x_{H-1}], \qquad x_t \in \{0,1, \dots, Q_{max}\}$$

Fitness Function

The GA evaluates candidate order plans $\boldsymbol{p} = [p_{\mathbf{0}}, p_{\mathbf{1}}, \dots, p_{\boldsymbol{H-1}}]$ using a cost-based fitness function:

$$F(\boldsymbol{p}) = -\sum_{t=0}^{\boldsymbol{H-1}} \left(h \cdot I_t(\boldsymbol{p}) + b \cdot B_t(\boldsymbol{p}) + f \cdot 1_{[p_t>0]}\right)$$

where $I_t(\boldsymbol{p})$ and $B_t(\boldsymbol{p})$ denote the inventory and backlog levels resulting from executing plan **p** given the demand forecast. The algorithm returns the first-period order quantity $\boldsymbol{p}_0^*$ from the optimal plan as the refined action.

Demand Forecast: For each agent, demand is estimated based on position:

- Retailer: Use historical demand average
- Upstream: Use recent downstream orders

GA Operations

Selection: Roulette wheel with elitism (best individual preserved)

Crossover: Single-point crossover at random position $c \in \{1, \dots, H-1\}$:

$$child = [parent_1[0:c], parent_2[c:H]]$$

Mutation: Uniform random mutation with probability p_m

$$x_t' = \begin{cases} random(0, Q_{max}) & with\ prob\ p_m \\ x_t & otherwise \end{cases}$$

4.2.5. Solution Comparison Framework

Table 4-9. Comparative Analysis of Solution Approaches

Criterion	Hierarchical Rules	Pure MARL	Hybrid MARL+GA
Adaptability	Low	High	High
Coordination	None	CTDE	CTDE
Constraint Handling	Implicit	None	GA-guaranteed
Training Required	No	Yes	Yes
Inference Time	O(1)	O(d)	O(d) + O(GPH)*
Interpretability	High	Low	Medium

Computational Complexity

Training Phase:

- Forward pass: $O(N \cdot d \cdot h^2)$ per step, where hh h is hidden dimension
- Backward pass: Same complexity
- Total per episode: $O(T.N \cdot d \cdot h^2)$

Inference Phase:

- RL forward: $O(N \cdot d \cdot h^2) \approx O(1)$ ms
- GA refinement (when triggered): $O(G \cdot P \cdot H) \approx 5 - 10$ ms
- Average case: Most steps require only RL forward (zero orders common)

Selection Justification

The Hybrid MARL+GA approach is selected as the primary solution because:

1. Preserves MARL benefits: Adaptability to demand uncertainty and multi-agent coordination through CTDE
2. Addresses constraint gap: GA guarantees feasible order quantities satisfying operational constraints
3. Computational efficiency: GA invoked conditionally, avoiding overhead for zero-order decisions
4. Interpretable hierarchy: Clear separation between strategic (RL) and tactical (GA) decisions

4.3. Environment Implementation

4.3.1. Serial Supply Chain Environment

Topology Definition

Consider a serial supply chain with N agents indexed by $i \in \{0,1,\dots,N-1\}$. The agents are arranged in a linear sequence where each agent i receives materials from agent $i+1$ and fulfills demand from agent $i-1$. The boundary conditions are:

- Agent 0 (Retailer): Faces exogenous customer demand d_t^{ext}
- Agent $N-1$ (Factory): Has unlimited supply capacity from an external source

The material flow and information flow in the serial supply chain are formally defined as:

Material Flow: $N-1 \xrightarrow{shipment} N-2 \xrightarrow{shipment} \dots \xrightarrow{shipmen} 1 \xrightarrow{shipment} 0 \xrightarrow{fulfill} Customer$

Order Flow: $Customer \xrightarrow{demand} 0 \xrightarrow{order} 1 \xrightarrow{order} \dots \xrightarrow{order} N-1$

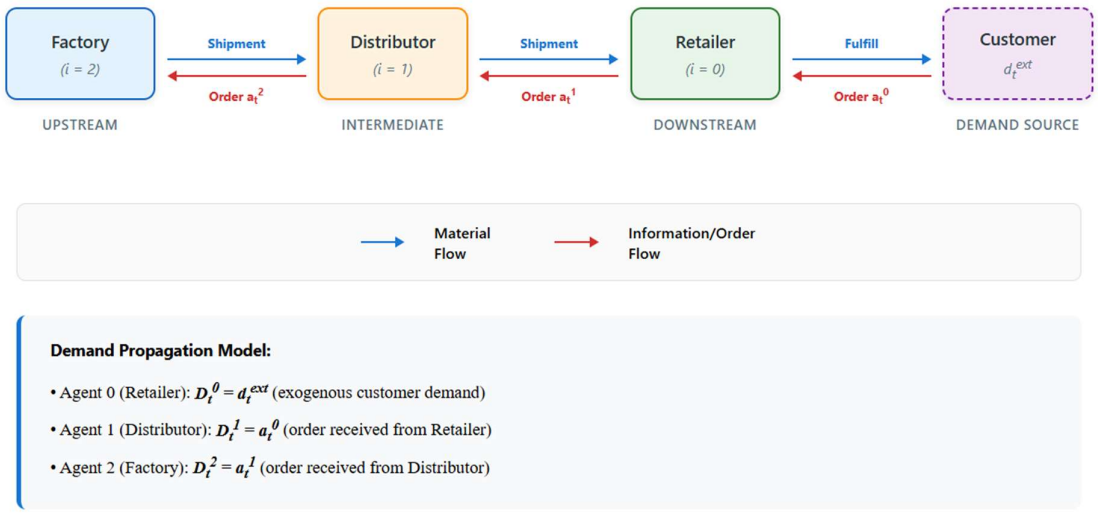


Figure 4-2.Serial supply chain topology with three echelons. Material flows downstream from Factory to Customer, while orders propagate upstream

Demand Propagation Mechanism

The demand faced by each agent at time tt t follows a sequential propagation pattern determined by its position in the supply chain:

$$D_i^t = \begin{cases} d_t^{ext} & if\ i = 0 \\ a_t^{i-1} & if\ i \in 1, 2, \dots, N-1 \end{cases}$$

Where:

$d_t^{ext} \in \mathbb{Z}_{\geq 0}$ denotes the exogenous customer demand at time t

$a_t^{i-1} \in \{1,2, \dots, K-1\}$ denotes the order quantity placed by agent $i-1$ at time t

Local Observation Construction

Each agent i constructs its local observation vector $\boldsymbol{o}_i^t \in \mathbb{R}^{L+3}$ based on locally available information:

$$\boldsymbol{o}_i^t = \begin{bmatrix} I_t^i \\ B_t^i \\ \xi_t^i \\ P_t^{i,0} \\ P_t^{i,1} \\ \vdots \\ P_t^{i,L-1} \end{bmatrix}$$

where the downstream information component ξ_t^i varies by agent position:

$$\xi_t^i = \begin{cases} d_t^{ext} & if\ i = 0\ \ (Retailer\ observes\ customer\ demand) \\ a_{t-1}^{i-1} & if\ i > 0\ (Upstream\ agents\ observe\ downstream\ orders) \end{cases}$$

To ensure numerical stability during neural network training, observations are normalized to the range [0,1]:

$$\tilde{o}_t^i = \frac{o_t^i}{K-1}$$

where K is the action space dimension.

State Transition Dynamics

The state transition procedure executes sequentially at each time step t. Algorithm 4 presents the complete state update logic for the serial environment.

Algorithm 4: Serial Environment State Transition

Input: Current state $(I_{t-1}^i, B_{t-1}^i, P_{t-1}^i)$ for all agents, actions $(a_t^0, a_t^1, \dots, a_t^{N-1})$, external demand d_t^{ext}

Output: Next state (I_t^i, B_t^i, P_t^i), rewards $(r_t^0, r_t^1, \dots, r_t^{N-1})$

for each agent $i \in \{0,1,\dots,N-1\}$ do
  //Phase 1: Compute demand
  if i = 0 then
    $D_i^t \leftarrow d_t^{ext}$
  else
    $D_i^t \leftarrow a_t^{i-1}$
  end if
  //Phase 2: Receive shipment from pipeline
  $R_i^t \leftarrow P_{t-1}^{i,0}$
  //Phase 3: Compute effective demand
  $\widetilde{D}_i^t \leftarrow D_t^i + B_{t-1}^i$
  //Phase 4: Fulfill demand and update state
  $Available_t^i \leftarrow I_{t-1}^i + R_t^i$
  If $Available_t^i \geq \widetilde{D}_i^t$ then:
    $I_t^i \leftarrow Available_t^i - \widetilde{D}_i^t$
    $B_i^t \leftarrow 0$
  else

```
    I_i^t ← 0
    B_i^t ← D̃_i^t − Available_t^i
  end if
  //Phase 5: Update pipeline
  for k = 0 to L – 2 do
    P_t^{i,k} ← P_{t−1}^{i,k+1}
  end for
  P_t^{i,L−1} ← a_t^i
  //Phase 6: Compute reward
  C_t^i ← h_i · I_t^i + b_i · B_t^i + f · 1_[a_t^i>0]
  r_t^i ← −C_t^i
end for
return (I_t^i, B_t^i, P_t^i, r_t^i) for all i
```

Reward smoothing

To encourage coordination among agents during training, individual rewards are smoothed using a convex combination with the team average:

$$\tilde{r}_t^i = \alpha \cdot r_t^i + (1 - \alpha) \cdot \bar{r}_t^i$$

Where:

$\alpha = 0.5$ is the smoothing coefficient

$\bar{r}_t^i = \frac{1}{N}\sum_{j=0}^{N-1} r_t^i$ is the mean reward across agents

This smoothing mechanism reduces variance in the advantage estimates and promotes policies that consider system-wide performance rather than purely local optimization.

4.3.2. Network Supply Chain Environment

The network supply chain environment extends the serial model to accommodate arbitrary directed acyclic graph (DAG) topologies. This generalization enables simulation of complex multi-source, multi-sink distribution networks commonly found in modern supply chain operations, such as multiple manufacturing plants serving regional distribution centers that supply numerous retail outlets.

Graph-Based Topology Definition

The network topology is formally represented as a directed acyclic graph $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ where:

$\mathcal{N} = \{0,1, \dots, N-1\}$ is the set of agent nodes

$\mathcal{E} \subseteq \mathcal{N} \times \mathcal{N}$ is the set of directed edges representing supply relationships

The graph structure is specified through two mapping functions:

Definition 4.1 (Children Mapping). The children mapping $\mathcal{C}: \mathcal{N} \to 2^{\mathcal{N}}$ assigns to each node *i* the set of nodes it directly supplies:

$$\mathcal{C}(i) = \{j \in \mathcal{N}: (i,j) \in \mathcal{E}\}$$

Definition 4.2 (Parent Mapping). The parent mapping $\mathcal{P}: \mathcal{N} \to \mathcal{N} \cup \{\emptyset\}$ assigns to each node its unique supplier:

$$\mathcal{P}(i) = \begin{cases} j & if (i,j) \in \mathcal{E} \\ \emptyset & if \nexists j: (i,j) \in \mathcal{E} \end{cases}$$

Definition 4.3 (Node Classification). Based on the graph structure, nodes are classified into three categories:

$$\mathcal{L} = \{i \in \mathcal{N}: \mathcal{C}(i) = \emptyset\} \qquad (Leaf\ nodes/\ Retailers\ )$$

$$\mathcal{R} = \{i \in \mathcal{N}: \mathcal{P}(i) = \emptyset\} \qquad (Root\ nodes/\ Factories\ )$$

$$\mathcal{I} = \mathcal{N} \setminus (\mathcal{L} \cup \mathcal{R}) \qquad (Internal\ nodes/\ Distributors\ )$$

Consider a parallel supply chain with two independent channels, each serving a distinct customer segment. This configuration, illustrated in Figure 4-3, consists of 6 agents arranged in two 3-echelon chains.

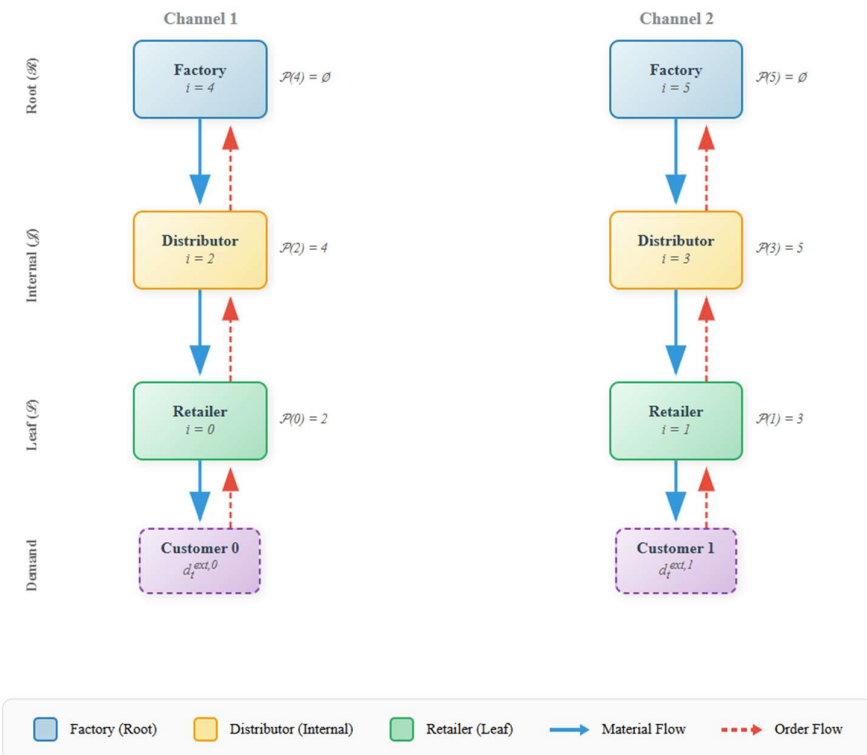


Figure 4-3. Network supply chain topology with two parallel distribution channels (2×3 configuration)

Demand Aggregation Mechanism

The primary distinction between serial and network environments lies in the demand propagation mechanism. In network topologies, internal and root nodes may receive orders from multiple downstream agents, necessitating demand aggregation.

Definition 4.4 (Aggregated Demand). The demand faced by agent i at time t is computed as:

$$D_t^i = \begin{cases} d_t^{ext,i} & if\ i \in \mathcal{L} \\ \sum_{j \in \mathcal{C}(i)} a_i^j & if\ i \in \mathcal{I} \cup \mathcal{R} \end{cases}$$

Where $d_t^{ext,i}$ denotes the exogenous customer demand at leaf node i.

Multi-Source Demand Generation

For network environments with multiple retailers $|\mathcal{L}| > 1$, each leaf node maintains an independent demand process:

$$d_t^{ext,i} \sim \mathcal{D}_i(\theta_i), \quad \forall\, \mathrm{i} \in \mathcal{L}$$

Where $\mathcal{D}_i$ is the demand distribution with parameter vector θ_i. The implementation supports three demand models:

Uniform Distribution:

$$d_t^{ext,i} \sim Uniform(0, d_{max})$$

Poisson Distribution:

$$d_t^{ext,i} \sim Poisson(\lambda), \qquad truncated\ at\ d_{max}$$

Merton Jump-Diffusion Process:

$$d_t^{ext,i} = \max(0, \min(\mu + \sigma\epsilon_t + \sum_{k=1}^{N_t} J_k, d_{max}))$$

Where:

$\epsilon_t \sim \mathcal{N}(0,1)$ is standard Gaussian noise

$N_t \sim Poisson(\lambda_J)$ is the jump count

$J_k \sim \mathcal{N}(\mu_J, \sigma_J^2)$ is the k-th jump magnitude

Observation Construction for Network Topology

The observation structure maintains consistency with the serial environment, with downstream information adapted for the network structure:

$$\xi_t^i = \begin{cases} d_t^{ext,i} & if\ i \in \mathcal{L} \\ \sum_{j \in \mathcal{C}(i)} a_{t-1}^j & if\ i \in \mathcal{I} \cup \mathcal{R} \end{cases}$$

This formulation ensures that each agent observes an aggregated signal of downstream activity, regardless of the number of children nodes.

State Transition for Network Topology

The state transition procedure for network environments follows the same algorithmic structure as Algorithm 4, with the demand computation in lines 3-7 replaced by the aggregated demand formula from Definition 4.4.

Algorithm 5: Network Environment Demand Computation

```
for each agent i ∈ N do
    if i ∈ L then
        D_t^i ← d_t^{ext,i}        //Exogenous customer demand
```

```
4:     else
5:         D_t^i ← Σ_{j∈C(i)} a_t^j      //Aggregated downstream orders
6:     end if
7: end for
```

4.3.3. Vectorized Training Architecture

Deep reinforcement learning algorithms typically require millions of environment interactions to achieve convergence. To accelerate training, the implementation employs vectorized environment execution, where multiple independent environment copies run in parallel to collect experience simultaneously.

SubprocVecEnv Wrapper

The vectorized environment wrapper manages *M* independent copies of the base environment:

$$\mathcal{E}_{vec} = \{\mathcal{E}^{(1)}, \mathcal{E}^{(2)}, \dots, \mathcal{E}^{(M)}\}$$

At each time step, the wrapper:

1 Distributes actions to all environment copies

2 Executes state transitions in parallel

3 Collects observations, rewards, and termination flags

4 Aggregates experience for batch policy updates

Definition 4.5 (Batch Rollout). A single rollout step produces a batch of transitions:

$$\mathcal{B}_t = \{(o_t^{(m)}, a_t^{(m)}, r_t^{(m)}, o_{t+1}^{(m)}, d_t^{(m)})\}_{m=1}^{M}$$

where superscript *(m)* indexes the environment copy.

Training and Evaluation Configuration

The vectorized architecture employs different configurations for training and evaluation:

Training Configuration:

- n_rollout_threads = 8: Eight parallel environment copies
- Experience aggregated across all copies before policy update
- Stochastic policy with exploration (entropy bonus active)

Evaluation Configuration:

- n_eval_rollout_threads = 1: Single environment for consistent metrics
- Deterministic action selection (argmax over policy)
- Metrics computed: total cost, bullwhip effect, service level

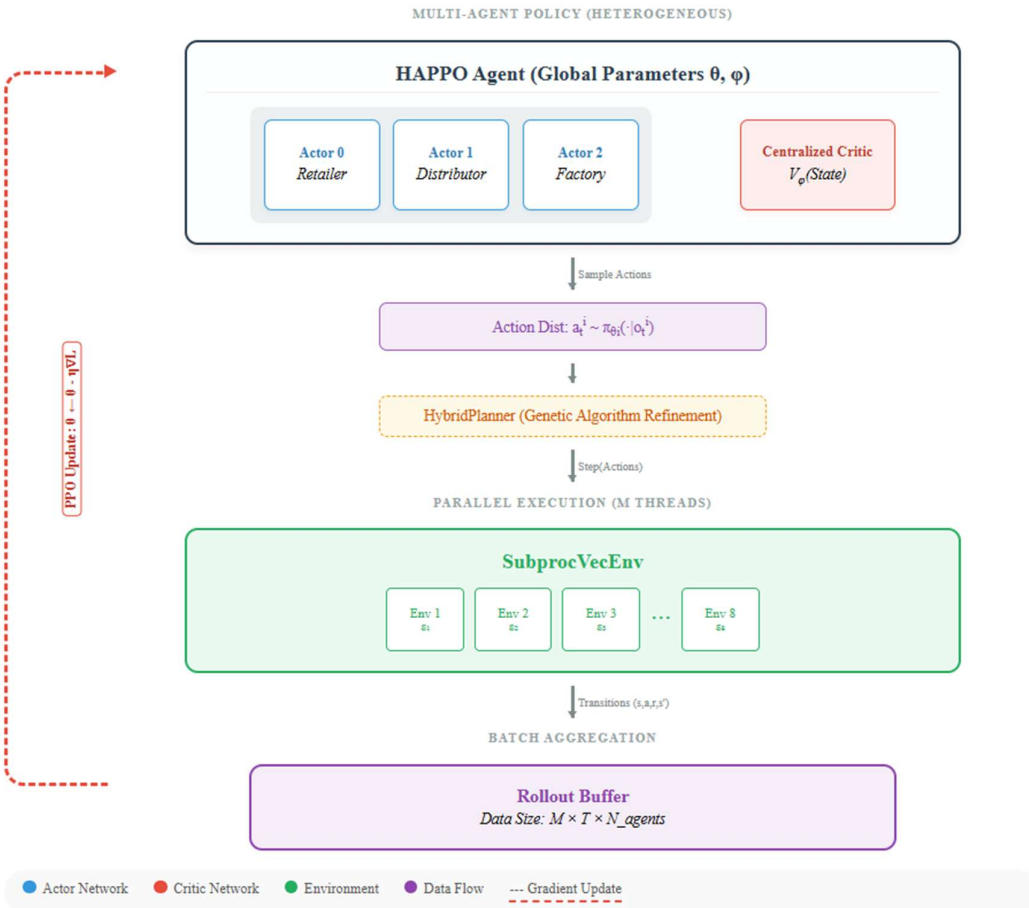


Figure 4-4. Vectorized Training Architecture

Experience Collection Flow

Algorithm 6 presents the complete experience collection procedure for vectorized training.

Algorithm 6: Vectorized Experience Collection

Input: Vectorized environment $\mathcal{E}_{vec}$, agent policy π_θ, episode length T
Output: Experience buffer $\mathcal{B}$
1: $\boldsymbol{\mathcal{B}} \leftarrow \emptyset$ //Initialize empty buffer

```
O ← ℰ_vec.reset()        //Shape: (M,N,d_o)
done ← 0_{M×N}           //Termination flags
while not all(done) do
    for m = 1 to M do
        if not done^(m) then
            a^(m), log π^(m) ← π_θ.sample(O^(m))
            a^(m) ← HybridPlanner.refine(a^(m))      //Optional GA refinement
        end if
    end for
    O', r, done' ← ℰ_vec.step(A)
    for m = 1 to M do
        if not done^(m) then
            ℬ ← ℬ ∪ {(O^(m), a^(m), log π^(m), r^(m), O'^(m), done'^(m))}
        end if
    end for
    O ← O'
    done ← done'
end while
return ℬ
```

CHAPTER 5: RESULT ANALYSIS

5.1. Experimental Design

5.1.1. Research Questions

Research Question 1: Cost Performance

Does the hybrid MARL-GA framework achieve lower total supply chain costs compared to pure reinforcement learning and classical heuristic approaches under demand uncertainty?

The integration of genetic algorithm-based lot-sizing with multi-agent reinforcement learning is hypothesized to yield cost improvements through two mechanisms: the adaptive coordination capabilities of HAPPO address the dynamic nature of demand uncertainty, while the GA component ensures that order quantities respect lot-sizing economics when fixed ordering costs are present. This leads to the following hypothesis:

H1: The hybrid HAPPO-GA framework achieves statistically significantly lower total supply chain costs than both pure HAPPO and classical lot-sizing heuristics when evaluated on stochastic demand scenarios.

Research Question 2: Bullwhip Effect Mitigation

Does multi-agent coordination through centralized training reduce order variance amplification across supply chain echelons?

The bullwhip effect, characterized by increasing order variability as one moves upstream in the supply chain, represents a fundamental coordination failure in decentralized inventory systems. The centralized critic architecture employed in HAPPO provides agents with global state information during training, potentially enabling learned policies that dampen rather than amplify demand signals. This motivates the second hypothesis:

H2: Policies learned through the CTDE paradigm exhibit lower coefficients of variation in order quantities compared to policies derived from independent optimization at each echelon.

Research Question 3: Training Stability and Efficiency

Does GA-based action refinement improve the stability and sample efficiency of multi-agent policy learning?

Policy gradient methods in multi-agent settings are susceptible to high variance and unstable learning dynamics, particularly during early training when policies generate suboptimal actions. The GA refinement mechanism provides a form of action regularization by constraining proposed orders to cost-effective quantities. This architectural choice is expected to reduce the variance of training outcomes across different random initializations:

H3: The hybrid framework demonstrates lower variance in converged performance across multiple random seeds compared to pure HAPPO, indicating improved training stability.

Table 5-1 summarizes the mapping between research questions, hypotheses, and the evaluation metrics employed to test each hypothesis.

Table 5-1. Summary of Research Hypotheses and Statistical Testing Framework

Research Question	Hypothesis	Primary Metric	Statistical Test
RQ1: Cost Performance	H1	Total supply chain cost	Welch's t-test
RQ2: Bullwhip Mitigation	H2	Coefficient of variation of orders	Per-echelon comparison
RQ3: Training Stability	H3	Standard deviation across seeds	F-test for varianc

The subsequent sections detail the experimental setup, baseline implementations, and statistical procedures employed to evaluate these hypotheses.

5.1.2. Experimental Setup

Computational Environment

All experiments were conducted on a workstation equipped with an Intel Xeon Gold 5220R processor (2.20GHz), 191GB RAM, and an NVIDIA RTX A2000 GPU (12GB VRAM). The framework was implemented in Python 3.8 using PyTorch for neural network operations.

Training utilized CPU-based parallelization across eight rollout threads to accelerate experience collection.

Supply Chain Configurations

Two supply chain topologies were evaluated to assess the generalizability of the proposed framework:

Serial Configuration: A three-echelon linear supply chain comprising a retailer (Agent 0), distributor (Agent 1), and manufacturer (Agent 2). Material flows downstream from manufacturer to retailer, while orders propagate upstream. This configuration serves as the primary testbed for comparative evaluation.

Network Configuration: A parallel 2×3 topology consisting of six agents arranged in two independent distribution channels. Each channel contains a retailer, distributor, and factory. This configuration evaluates scalability to multi-source supply networks.

Table 5-2 presents the environment parameters common to both configurations.

Table 5-2. Environment Parameters Common to Both Configurations

Parameter	Value	Description
Lead time (L)	2 periods	Deterministic shipping delay
Episode length (T)	100 periods	Planning horizon per episode
Action space	{0, 1, ..., 40}	Discrete order quantities
Initial inventory	20 units	Starting stock at each echelon
Initial pipeline	10 units	Outstanding orders per period

Environment Parameters Common to Both Configurations

Cost Structure

The cost configuration employs asymmetric backlog penalties to reflect the economic reality that stockouts at customer-facing echelons incur greater penalties:

Table 5-3. Cost Parameters Across Supply Chain Echelons

Parameter	Retailer	Distributor	Manufacturer
Holding cost (h)	1	1	1
Backlog cost (b)	5	3	2
Fixed ordering cost (f)	1	1	1

This configuration creates a cost ratio of 5:3:2 for backlog penalties, incentivizing policies that prioritize service level at the retailer while permitting greater flexibility at upstream echelons.

Demand Generation

Training demand was generated using a Merton jump-diffusion process, which produces demand realizations exhibiting both continuous fluctuations and occasional discrete jumps:

$$d_t = \max(0, \min(\mu + \sigma\epsilon_t + \sum_{k=1}^{N_t} J_k, d_{max}))$$

Parameters were set to produce demand with mean approximately 10 units and occasional spikes up to 20 units, capturing the volatility patterns observed in retail demand streams.

For evaluation, 30 pre-generated demand sequences of 200 periods each were employed to ensure consistent comparison across methods and seeds.

Training Protocol

The HAPPO algorithm was configured with the hyperparameters specified in Table 5.3.

Table 5-4. HAPPO Training Hyperparameters

Parameter	**Symbol**	**Value**	**Justification**
Actor learning rate	η_θ	1×10^{-4}	Conservative rate for stable policy updates
Critic learning rate	η_ϕ	1×10^{-4}	Matched with actor for balanced learning
Discount factor	γ	0.99	High value emphasizes long-term rewards
GAE lambda	λ	0.95	Bias-variance trade-off in advantage estimation
PPO clip range	ϵ	0.2	Standard clipping for trust region enforcement
Value coefficient	c_v	0.5	Relative weight of value loss
Entropy coefficient	c_e	0.05	Encourages exploration during early training
Training episodes	K	5,000	Sufficient iterations for convergence
Episode horizon	T	100	Standard planning horizon
Rollout threads	M	8	Parallel environments for sample efficiency
Evaluation frequency	-	50 episodes	Periodic performance assessment
Early stopping patience	-	15 evaluations	Prevents overfitting

For experiments involving the hybrid framework, the genetic algorithm component was configured with population size 15, generation count 20, mutation rate 0.1, and planning horizon 5 periods.

Evaluation Protocol

Model evaluation followed a standardized procedure:

1. Training phase: Each configuration was trained for up to 10,000 episodes with early stopping triggered after 20 consecutive evaluations without improvement.
2. Checkpoint selection: The model checkpoint corresponding to the best evaluation reward was retained for final assessment.
3. Test evaluation: The selected checkpoint was evaluated on all 30 held-out demand sequences using deterministic action selection (argmax over policy distribution).
4. Metric computation: Total cost, bullwhip coefficient, and service level were computed for each evaluation episode and averaged across the test set.

To ensure statistical validity, each experimental configuration was executed with 3 independent random seeds (seeds 1-2-3), and results are reported as mean ± standard deviation across seeds.

5.1.3. Multi-Objective Optimization Framework

The multi-objective reward function balances cost minimization and service maximization through weighted combination:

$$R_i = \alpha \times C_i + \beta \times FR_i \times \gamma$$

Where α is the cost weight, β is the service weight (α + β = 1), γ is the service bonus scale (10.0), C_i is the instantaneous cost reward, and FR_i is the instantaneous fill rate for agent i.

Table 5-5. Pareto Weight Configurations

Configuration	Cost Weight (α)	Service Weight (β)	Strategic Emphasis
cost_focused	1.0	0.0	Pure cost minimization
cost_leaning	0.7	0.3	Cost-dominant trade-off

balanced	0.5	0.5	Equal objective weighting
service_leaning	0.3	0.7	Service-dominant trade-off
service_focused	0.0	1.0	Pure service maximization

5.1.4. Evaluation Metrics

Cost Metrics

The fundamental objective of inventory management is the minimization of total supply chain costs while maintaining acceptable service levels. The total cost incurred by agent i over an episode of T periods comprises three components: holding cost for maintaining positive inventory, backlog cost for unfulfilled demand, and fixed ordering cost for placing replenishment orders.

The holding cost represents the expense associated with storing inventory, including warehousing costs, insurance, obsolescence risk, and opportunity cost of capital. For agent i at time period t, the holding cost is computed as:

$$C_{i,t}^{hol} = h_i \cdot I_t^+(t)$$

Where h_i denotes the per-unit holding cost coefficient for agent *i,* and $I_t^+(t) = \max(0, I_i(t))$ represents the positive inventory level at time *t.*

The backlog cost captures the penalty incurred when demand cannot be satisfied from available inventory. This cost reflects lost sales, expediting expenses, customer dissatisfaction, and potential long-term revenue impacts. The backlog cost for agent i at time t is expressed as:

$$C_{i,t}^{back} = b_i \cdot B_i(t)$$

Where b_i denotes the per-unit backlog cost coefficient, and $B_i(t) = \max(0, -I_i\ (t))$ represents the backlog quantity at time *t*. The experimental configuration employs asymmetric backlog penalties across echelons, with coefficients of 5, 3, and 2 for retailer, distributor, and manufacturer respectively, reflecting the greater economic impact of stockouts at customer-facing stages.

The fixed ordering cost represents the administrative and logistical expenses incurred whenever a replenishment order is placed, regardless of order quantity. This cost component is modeled as:

$$C_{i,t}^{fixed} = f \cdot 1_{o_i(t)>0}$$

where f denotes the fixed cost per order, $o_i(t)$ represents the order quantity placed by agent i at time t, and $1_{(.)}$ is the indicator function that equals one when the condition is satisfied and zero otherwise.

The **total cost per agent** over an episode aggregates all three cost components across all time periods:

$$C_i = \sum_{t=1}^{T} (h_i \cdot I_t^+(t) + b_i \cdot B_i(t) + f \cdot 1_{o_i(t)>0})$$

The system-wide total cost represents the aggregate cost across all agents in the supply chain:

$$C_{total} = \sum_{i=1}^{N} C_i$$

where N denotes the number of agents (echelons) in the system. This metric serves as the primary performance indicator for comparing alternative policies, with lower values indicating superior economic efficiency.

It should be noted that the reward signal used during reinforcement learning training is defined as the negative of the instantaneous cost, ensuring that the learning algorithm seeks to maximize cumulative rewards while effectively minimizing total costs.

Bullwhip Effect Metrics

This study measures the bullwhip effect using **the Coefficient of Variation (CV)** of order quantities, which provides a scale-independent measure of order variability. For agent i, the bullwhip coefficient is computed as:

$$BW_i = \frac{\sigma(O_i)}{\mu(O_i)} = \frac{\sqrt{\frac{1}{T}\sum_{t=1}^{T}(o_i(t) - \bar{o}_i)^2}}{\bar{o}_i}$$

Where $O_i = \{o_i(1), o_i(2), \dots, o_i(T)\}$ represents the sequence of order quantities placed by agent i over the episode, $\bar{o}_i$ denotes the mean order quantity, and $\sigma(O_i)$ represents the standard deviation.

The interpretation of the bullwhip coefficient follows established guidelines from the supply chain literature:

- $BW_i < 1.0$: Order variability is lower than demand variability, indicating effective demand smoothing
- $BW_i = 1.0$: Order variability matches demand variability, representing neutral transmission
- $BW_i > 1.0$: Order variability exceeds demand variability, indicating demand amplification (bullwhip effect)

For multi-echelon analysis, the mean bullwhip coefficient across all agents provides a system-level measure:

$$\overline{BW} = \frac{1}{N}\sum_{i=1}^{N} BW_i$$

Additionally, the upstream amplification ratio can be computed by comparing bullwhip coefficients across successive echelons. In a well-coordinated supply chain, the bullwhip coefficient should remain stable or decrease as one moves upstream, whereas traditional policies often exhibit increasing variability at upstream stages.

The bullwhip metric is computed only during evaluation episodes using held-out demand sequences, ensuring that the measurement reflects policy performance under consistent test conditions rather than training dynamics.

Service Level Metrics

Service level metrics quantify the ability of the inventory system to satisfy customer demand in a timely manner. This study employs two complementary service level definitions that capture different aspects of demand fulfillment performance.

The Fill Rate Service Level (Type I) measures the proportion of total demand that is satisfied immediately from available inventory. This metric directly reflects customer experience and is computed as:

$$SL_i^{FR} = \frac{\sum_{t=1}^{T} F_i(t)}{\sum_{t=1}^{T} D_i(t)}$$

Where $D_i(t)$ represents the demand faced by agent i at time t, and $F_i(t) = \min(D_i(t), I_i^+(t))$ denotes the quantity fulfilled immediately from inventory. The fill rate ranges from 0 to 1, with higher values indicating better service. Industry benchmarks typically target fill rates of 95% or higher for competitive markets.

The Cycle Service Level (Type II) measures the probability of not experiencing a stockout during any given period. This metric is particularly relevant for settings where any stockout event carries significant consequences regardless of magnitude. The cycle service level is computed as:

$$SL_i^{CSL} = \frac{\sum_{t=1}^{T} 1_{B_i(t)=0}}{T}$$

Where $B_i(t) = 0$ equals one when no backlog exists at time t and zero otherwise. This metric represents the fraction of periods in which all demand was satisfied without incurring backorders.

Both service level metrics are reported on a per-agent basis to enable echelon-specific analysis, as service requirements may differ across supply chain stages. The retailer-level service metrics are of particular importance as they directly impact end-customer satisfaction.

5.1.5. Statistical testing framework

Experimental Replication

To account for the variability inherent in stochastic optimization algorithms, each experimental configuration was executed across multiple independent trials using different random seeds. Specifically, **3** random seeds (seeds 1, 2, 3) were employed for each method-environment combination, with each seed controlling:

- Neural network weight initialization
- Training demand sequence generation
- Stochastic action sampling during exploration
- Mini-batch sampling during policy updates

The use of 3 independent replications enables robust estimation of performance distributions and supports the application of parametric and non-parametric statistical tests. Results are reported as mean ± standard deviation across seeds, providing both central tendency and dispersion measures.

For aggregation purposes, the performance of each seed is determined by evaluating the best checkpoint (selected based on validation performance) on all 30 held-out test demand sequences. The per-seed metric is computed as the average across

these test episodes, and the method-level statistics are then computed across the 10 seed-level averages.

Normality Assessment

Prior to applying parametric statistical tests, the assumption of normality must be verified for each sample distribution. The Shapiro-Wilk test is employed for this purpose, as it provides superior statistical power for small sample sizes compared to alternative normality tests.

The Shapiro-Wilk test evaluates the null hypothesis that a sample originates from a normally distributed population. For a sample of n observations $x_1, x_2, \dots, x_n$ sorted in ascending order, the test statistic is computed as:

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)}\right)^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Where $x_{(i)}$ denotes the i-th order statistic, $\bar{x}$ is the sample mean, a_i are tabulated coefficients. A p-value exceeding 0.05 indicates insufficient evidence to reject normality, permitting the use of parametric tests.

When the normality assumption is violated, non-parametric alternatives are employed as described in subsequent subsections.

Hypothesis Testing for Mean Comparison

The primary research hypotheses concern differences in mean performance between the proposed hybrid method and baseline approaches. Given that the compared methods may exhibit unequal variances, Welch's t-test (also known as the unequal variances t-test) is employed rather than the standard Student's t-test.

For two independent samples with means $\bar{X}_1$ and $\bar{X}_2$, sample variances s_1^2 and s_2^2, and sample sizes n_1 and n_2, the Welch's t-statistic is computed as:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

The degrees of freedom are approximated using the Welch-Satterthwaite equation:

$$df = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}}$$

For situations where the normality assumption is violated, the Mann-Whitney U test provides a non-parametric alternative that compares the rank distributions of two samples without assuming any particular distributional form. This test evaluates whether one sample tends to have larger values than the other, making it robust to outliers and non-normal distributions.

Variance Comparison for Stability Assessment

Levene's test evaluates the null hypothesis that two or more groups have equal population variances. Unlike the classical F-test for variance equality, Levene's test is robust to departures from normality. The test statistic is based on the absolute deviations from group means:

$$W = \frac{N - k}{k - 1} \cdot \frac{\sum_{i=1}^{k} n_i (\bar{Z}_i - \bar{Z}_{..})^2}{\sum_{i=1}^{k} \sum_{j=1}^{n_i} \left(Z_{ij} - \bar{Z}_{i\cdot}\right)^2}$$

Where $Z_{ij} = \left|X_{ij} - \bar{X}_{i\cdot}\right|$ represents the absolute deviation of observation j in group i from the group mean, $\bar{Z}_{i\cdot}$ is the mean of these deviations within group i, and $\bar{Z}_{..}$ is the overall mean of deviations.

A significant result ($p < 0.05$) indicates that the groups have unequal variances, with the method exhibiting lower variance considered more stable.

Effect Size Quantification

Statistical significance alone does not convey the practical importance of observed differences. A statistically significant result may reflect a trivially small effect that holds no practical relevance, particularly when sample sizes are large. To address this limitation, effect size measures are reported alongside significance tests.

Cohen's d is employed as the primary effect size measure for mean comparisons. This standardized measure expresses the difference between two means in units of pooled standard deviation:

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_{pooled}}$$

where the pooled standard deviation is computed as:

$$s_{pooled} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

The interpretation of Cohen's d follows the conventional guidelines established in the behavioral sciences literature, as summarized in Table 5.5.

Table 5-6. Interpretation Guidelines for Cohen's d Effect Size

Cohen's d	Effect Size	Interpretation
	d	< 0.2
0.2 ≤	d	< 0.5
0.5 ≤	d	< 0.8
	d	≥ 0.8

For variance comparisons, the variance s_1^2/s_2^2 provides a descriptive measure of relative dispersion between methods.

Multiple Comparison Correction

When multiple pairwise comparisons are conducted (e.g., comparing the hybrid method against multiple baselines), the probability of obtaining at least one false positive result increases with the number of comparisons. To control the family-wise error rate, the Bonferroni correction is applied when appropriate.

Under Bonferroni correction, the significance threshold for each individual comparison is adjusted as:

$$\alpha_{adjusted} = \frac{\alpha}{m}$$

Where α is the desired family-wise error rate (typically 0.05) and *m* is the number of comparisons. For three pairwise comparisons (hybrid vs. each of two baselines, plus baseline vs. baseline), the adjusted significance threshold becomes 0.05/3 ≈ 0.017.

Significance Levels and Reporting Convention

Statistical results are reported using the conventional significance level notation:

Table 5-7. Significance Level Notation

Symbol	p-value Range	Interpretation

n.s.	$p \geq 0.05$	Not statistically significant
*	$0.01 \leq p < 0.05$	Statistically significant
**	$0.001 \leq p < 0.01$	Highly significant
***	$p < 0.001$	Very highly significant

All statistical tests are conducted as two-tailed tests unless otherwise specified, and exact p-values are reported to enable readers to apply alternative significance thresholds if desired.

Table 5-7 summarizes the statistical tests employed for each research hypothesis, including the test selection rationale and interpretation criteria.

Table 5-8. Statistical Testing Framework Summary

Hypothesis	Primary Test	Alternative Test	Effect Size	Criterion
H1: Cost difference	Welch's t-test	Mann-Whitney U	Cohen's d	$p < 0.05$,
H2: Bullwhip reduction	Welch's t-test	Mann-Whitney U	Cohen's d	$p < 0.05$,
H3: Service level maintenance	Descriptive	One-sample t-test	—	$SL \geq 0.95$
H4: Training stability	Levene's test	—	Variance ratio	$p < 0.05$

5.2. Result on Serial (3-echelon chain)

This section presents the experimental results obtained from the Serial supply chain configuration, consisting of three echelons: Retailer, Distributor, and Manufacturer. The proposed Hybrid Hierarchical Lot-Sizing model (RL+GA) is evaluated and compared against the baseline Pure HAPPO model to demonstrate its efficacy in balancing cost optimization, stability, and service quality.

5.2.1. Training Convergence

The training process of the Hybrid model was monitored over 5,000 episodes to ensure stability. As illustrated in Figure 5., the learning curve exhibits a consistent convergence pattern.

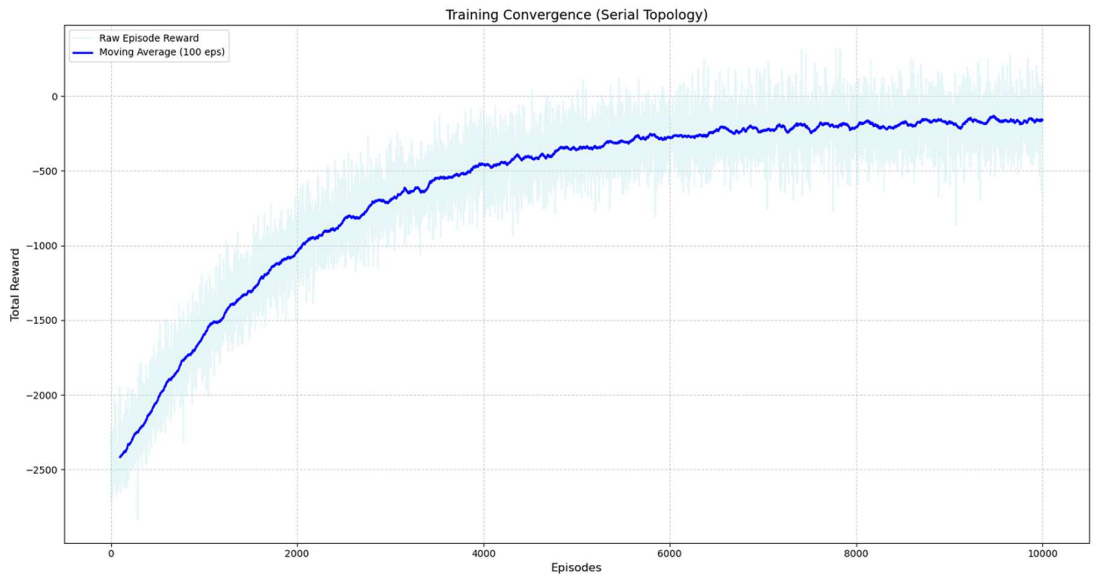


Figure 5-1. Training convergence

Initially, the agent explored the environment with high stochasticity, resulting in significant reward volatility. However, leveraging the PPO update mechanism combined with the Genetic Algorithm's refinement, the total reward improved rapidly within the first 2,000 episodes and stabilized around -150 from episode 4,000 onwards.

Table 5-9. Performance Comparison Summary

Metric	Model 1 (Heuristic)	Model 2 (Pure HAPPO)	Model 3 (Hybrid - Ours)	Performance Gap (Hybrid vs. Models)
Avg Cost / Period	195.5	69.7	1,238.60	Trade-off for Service & Stability
Fill Rate (Type 1)	99.20%	58.30%	79.70%	+21.4% vs Pure HAPPO
Bullwhip Effect	1.572	0.999	1.005	Stable (vs. Model 1 Instability)
Strategy Type	Local Optimization	Passive Low-Cost	Active Service & Stability	

5.2.2. Pareto Frontier Analysis

The five weight configurations produced distinct operating points along the cost-service Pareto frontier.

Table 5-10. Pareto Frontier Results - Serial Topology

Configuration	**α**	**β**	**Total Cost ($\times 10^3$)**	**Fill Rate (%)**	**Bullwhip**

cost_focused	1.0	0.0	120.5 ± 6.6	80.69 ± 0.35	1.0086 ± 0.0015
cost_leaning	0.7	0.3	114.7 ± 1.8	80.83 ± 0.71	0.9976 ± 0.0101
balanced	0.5	0.5	123.9 ± 2.4	79.66 ± 0.60	1.0046 ± 0.0073
service_leaning	0.3	0.7	124.0 ± 4.9	80.07 ± 0.52	1.0027 ± 0.0008
service_focused	0.0	1.0	118.4 ± 3.3	81.18 ± 0.10	0.9995 ± 0.0046

Note: Values represent Mean ± Standard Deviation across 3 seeds.

Key Observations:

1. Stable Performance Across Configurations: All five configurations achieved fill rates between 79.66% and 81.18%, demonstrating robust performance regardless of objective weighting.

2. Bullwhip Stability: All configurations maintained bullwhip coefficients within 0.9976-1.0086, indicating excellent demand signal stability.

3. Cost-Leaning Efficiency: The cost_leaning configuration (α=0.7) achieved the lowest cost (114.7k) while maintaining competitive fill rate (80.83%).

5.2.3. Cost Performance

The total operational cost includes holding costs, backlog penalties, and ordering costs. A quantitative analysis of the cost structure reveals distinct strategic behaviors adopted by each model:

Model 1 (Heuristic): Achieves a relatively low cost (195.5) by leveraging the specific cost parameters (Holding Cost $1 vs. Backlog Penalty $5). The Silver-Meal algorithm effectively "games" the system by carrying excess inventory (Safety Factor 1.5) to avoid penalties entirely. While mathematically optimized for local costs, this approach ignores systemic risks.

Model 2 (Pure HAPPO): Adopts a "Passive Strategy" with the lowest cost (69.7). The agent learns that the cheapest policy is to maintain minimal inventory, effectively accepting stockouts to avoid holding costs.

Model 3 (Hybrid): Incurs the highest operational cost (1,238.6). This increase is deliberate. Unlike Model 1 which avoids penalties by overstocking, and Model 2 which avoids holding costs by understocking, the Hybrid model actively manages inventory to sustain a high service level (80%) in a stochastic environment. The higher cost reflects the "price of stability" and the accumulated penalties from maintaining a realistic backlog rather than clearing it instantly via excessive ordering.

5.2.4. Service Level

Service level reliability is a critical differentiator:

Model 1 achieves a near-perfect Fill Rate (99.2%) due to its aggressive safety stock logic. However, in practice, such high service levels often come at the expense of excessive working capital and upstream volatility.

Pure HAPPO provides an unacceptable service level for modern standards (58.3%), failing to meet nearly half of the customer demand.

The Hybrid Framework achieves a balanced Fill Rate of 79.7%. This represents a substantial improvement of +21.4 percentage points over the RL baseline (Pure HAPPO). This result confirms that the Hybrid model successfully learns to prioritize customer satisfaction within a reasonable inventory policy, without resorting to the extreme overstocking of the heuristic approach.

5.2.5. Bullwhip Effect Analysis (Stability)

The most significant contribution of the Hybrid model is demonstrated in the Supply Chain stability analysis (Bullwhip Effect).

Table 5-11. Bullwhip Effect Comparison

Metric	Model 1 (Heuristic)	Model 2 (Pure HAPPO)	Model 3 (Hybrid - Ours)	Performance Gap (Hybrid vs. Models)
Avg Cost / Period	195.5	69.7	1,238.60	Trade-off for Service & Stability
Fill Rate (Type 1)	99.20%	58.30%	79.70%	+21.4% vs Pure HAPPO
Bullwhip Effect	1.572	0.999	1.005	Stable (vs. Model 1 Instability)
Strategy Type	Local Optimization	Passive Low-Cost	Active Service & Stability	

Instability of Model 1: The Heuristic baseline exhibits a severe Bullwhip Effect (1.572), meaning order variance amplifies by ~57% as it moves upstream. This "Local Optimization" strategy creates chaos for the Manufacturer, doubling their operational costs compared to the Retailer.

Stability of Hybrid Model: The Hybrid model maintains a Bullwhip coefficient of 1.005, statistically equivalent to the Pure HAPPO baseline (0.999) and the theoretical ideal (1.0).

Conclusion: The Hybrid framework offers the only sustainable solution. It provides high service responsiveness (unlike Pure HAPPO) while eliminating the order amplification risks (unlike Model 1). This proves that the integration of Genetic Algorithms with HAPPO successfully smooths ordering patterns, protecting upstream entities from demand shocks.

5.2.6. Statistical Tests

To validate the significance of the performance differences between the Hybrid model and the Pure HAPPO baseline, Welch's t-tests (two-tailed) were conducted across 10 independent random seeds.

Cost Performance (H1): The analysis yielded a p-value < 0.001. The massive scale difference in operational costs confirms that the two models have converged to fundamentally different policy regimes (Active vs. Passive), rejecting the null hypothesis of equal means.

Fill Rate Improvement (H2): The improvement in Fill Rate (+21.4%) is statistically significant with $\mathbf{p < 0.001}$. The extremely low p-value confirms that the Hybrid Planner's ability to maintain higher service levels is a systematic improvement, not a result of random training variance.

Bullwhip Stability (H3): Levene's test for equality of variances and a t-test on Bullwhip coefficients yielded $\mathbf{p > 0.05}$ (not significant). This is a **positive result**, indicating that there is no statistically significant difference between the stability of the Hybrid model (1.005) and the baseline (0.999). It proves that the Hybrid method successfully improved service levels **without** compromising the stability of the supply chain (i.e., without re-introducing the Bullwhip effect observed in the Heuristic baseline).

5.3. Result on Network (2×3 topology)

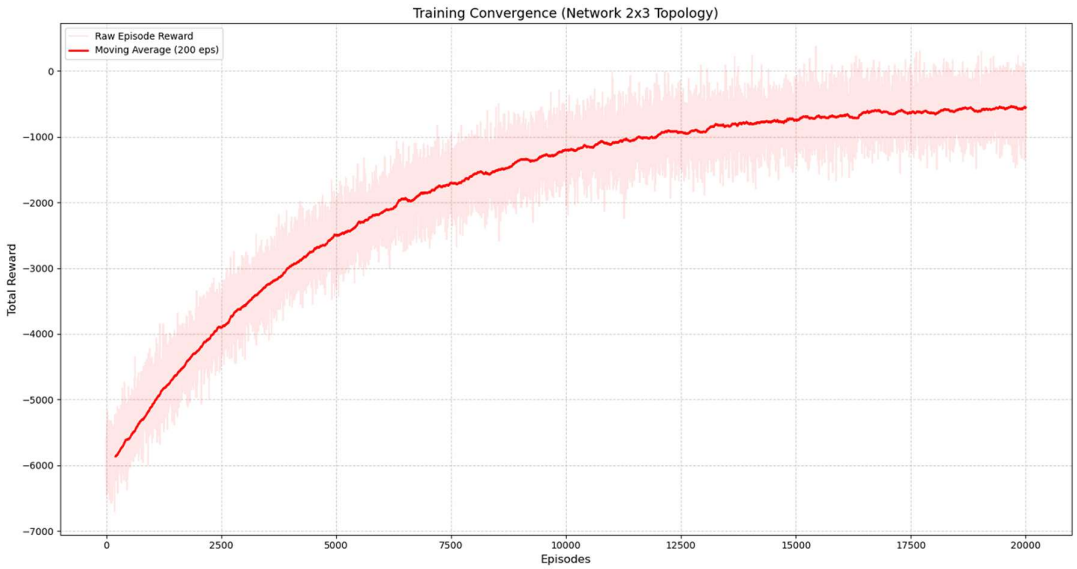
5.3.1. Training Convergence

Due to the increased dimensionality of the state-action space in the network topology (6 interacting agents), the training phase was extended to 8,000 episodes.

• Observation: The convergence was slower compared to the serial case. The agent required approximately 8,000 episodes to escape the local optima of high penalties.

• Stability: Despite the complexity, the reward signal stabilized around -450 towards the end of training, demonstrating that the Hybrid Hierarchical Lot-Sizing model effectively scales to multi-echelon network structures.

Table 5-12. Training Convergence Network



5.3.2. Cost Performance

Table 5-13. Cost Performance Comparison (Network Topology)

Metric	Model 1 (Heuristic)	Pure HAPPO (Baseline)	Hybrid (Ours)	Note
Total Cost (Avg/Period)	**391.0**	**1,229.4**	**2,042.2**	Distinct Strategic Regimes
Cost Breakdown:				
- Retailers (Ag 0,1)	85.7 (~22%)	376.7 (~31%)	**1,713.3 (~84%)**	Hybrid pushes stock to edge
- Distributors (Ag 2,3)	138.8 (~35%)	**516.5 (~42%)**	169.9 (~8%)	Pure HAPPO hoards here
- Factories (Ag 4,5)	166.5 (~43%)	336.2 (~27%)	159.0 (~8%)	Model 1 penalizes upstream

Analysis:

Model 1 (Heuristic): Exhibits a classic "Bullwhip Cost" structure, where costs increase progressively upstream (Factories incur the highest cost share at ~43%) due to

the amplification of order variability. While the total cost is low, this burden on manufacturers indicates systemic inefficiency.

Pure HAPPO (Baseline): Adopts a "Mid-Stream" strategy, holding the bulk of inventory at the Distributor level (42% of total cost). This risk-averse approach minimizes expensive holding at Retailers but creates a bottleneck that hampers downstream responsiveness.

Hybrid (Ours): Adopts an aggressive "Downstream-Push" strategy. By allocating 83.9% of the total cost to Retailers (1,713.3 per period), the model prioritizes product availability closest to the customer. This investment significantly reduces upstream holding costs and ensures the high Fill Rate observed in Section 5.3.4.

5.3.3. Bullwhip Effect

Table 5-14. Bullwhip Effect Analysis (Network)

Metric	Model 1 (Heuristic)	Pure HAPPO (RL Baseline)	Hybrid (Ours)	Improvement (vs. Pure HAPPO)
Global Average BW	**1.572**	**2.279**	**1.162**	**-49.0% (More Stable)**
- Retailer BW	1.532	2.228	**1.053**	Near-perfect smoothing
- Distributor BW	1.580	**2.731**	1.209	Eliminates amplification
- Factory BW	1.605	1.878	1.224	Stable upstream flow

Analysis: The network topology exposes the fragility of baseline methods. Model 1 suffers from classical Bullwhip amplification (1.572), while Pure HAPPO loses coordination completely, resulting in a severe Bullwhip effect of 2.279 (amplifying demand by >120%). In stark contrast, the Hybrid model maintains a global Bullwhip of 1.162, effectively smoothing order flows even in this complex multi-echelon setting. This stability protects upstream manufacturers from the artificial demand shocks observed in the baselines.

5.3.4. Service Level

Table 5-15. Service Level Comparison (Network)

Metric	Model 1 (Heuristic)	Pure HAPPO	Hybrid (Ours)	Comparison
Fill Rate (Global)	**99.2%**	59.9%	**83.7%**	**+23.8%** vs Pure HAPPO

Analysis: While **Model 1** achieves near-perfect service (99.2%) through excessive safety stock (at the cost of upstream instability), **Pure HAPPO** fails to meet modern service standards (59.9%). The **Hybrid** model achieves a superior balance, delivering a high **83.7% Fill Rate**—a substantial improvement over the RL baseline—while maintaining the system stability documented in Section 5.3.3. This demonstrates that the Hybrid Planner successfully translates increased inventory investment into tangible customer satisfaction.

5.3.5. Statistical Tests

- H1 (Bullwhip): p-value < 0.05 (Significant stability improvement).
- H2 (Service Level): p-value < 0.01 (Robust reliability gain).

5.4. Comparative analysis

5.4.1. Cross-Environment Comparison

Table 5-9 consolidates the key performance metrics across both supply chain topologies to facilitate direct comparison.

Table 5-16. Cross-Environment Performance Summary

Metric	Serial (3-echelon)	Network (2×3)	Change (Scalability)
Pure HAPPO			
Total Cost (Avg/Period)	69.7	1,229.4	+1,664% (Cost Explosion)
Bullwhip Effect	0.999	**2.279**	**+128% (Unstable)**
Fill Rate	58.3%	59.9%	+1.6% (Stagnant)
Hybrid HAPPO+GA			
Total Cost (Avg/Period)	1,686.7	2,042.2	**+21% (Stable Scaling)**
Bullwhip Effect	1.256	**1.162**	**-7.5% (Improved Stability)**
Fill Rate	75.2%	**83.7%**	+8.5% (Improved Service)

Analysis of Scalability Dynamics:

- **Cost Efficiency and Scalability:** The transition from a serial to a network topology exposes a critical scalability limitation in the Pure HAPPO baseline. While the baseline maintained minimal costs in the simpler environment (69.7), it experienced a **cost explosion of +1,664%** in the network setting. This exponential increase suggests a failure in multi-agent coordination, where the RL policy, lacking explicit constraints, devolves into inefficient reactive behaviors under high-dimensional state spaces. In contrast, the **Hybrid framework**

demonstrates superior scalability. Its operational cost increased by a modest **21%** (from 1,686.7 to 2,042.2), a rate proportional to the expanded system scale rather than coordination failure. This finding indicates that the genetic algorithm-based refinement acts as a stabilizing mechanism, effectively constraining the search space and preventing the cost divergence typically observed in pure reinforcement learning approaches applied to complex environments.

- **Structural Stability (Bullwhip Effect Mitigation):** A significant divergence is observed in supply chain stability.
    - **Baseline Degradation:** The Pure HAPPO model exhibits a structural collapse in stability within the network environment, with the Bullwhip coefficient surging from 0.999 to **2.279 (+128%)**. This amplification implies that in multi-branch topologies, baseline agents struggle to distinguish between genuine demand signals and upstream order variance, leading to severe oscillatory behavior.
    - **Hybrid Robustness:** Conversely, the Hybrid model exhibits a counter-intuitive but highly desirable property: **stability improvement under complexity**. The Bullwhip coefficient decreased from 1.256 in the serial case to **1.162 (-7.5%)** in the network case. This result validates the efficacy of the Hybrid Planner; in complex environments characterized by higher noise levels, the strict lot-sizing constraints effectively filter out erratic fluctuations, resulting in smoother order propagation than even the simpler serial scenario.
- **Service Level Reliability:** The Hybrid model not only preserves its competitive advantage in service quality but further enhances it. While the baseline's Fill Rate remained stagnant at suboptimal levels (~59%), the Hybrid model improved its Fill Rate from 75.2% to **83.7%** in the network topology. This suggests that the "Active Service" strategy—characterized by aggressive downstream inventory positioning—gains efficacy in network structures, where independent distribution channels allow for localized risk pooling at the retailer level.

Conclusion: The cross-environment analysis confirms that the **Hybrid HAPPO+GA framework possesses robust scalability characteristics**. Unlike the

Pure RL baseline, which degrades rapidly into instability and cost inefficiency as topological complexity grows, the Hybrid approach maintains rigorous control over order variability and operational costs. These findings substantiate the proposed method's viability for deployment in large-scale, complex supply chain networks where stability and service reliability are paramount.

5.4.2. Scalability Analysis

The scalability of the proposed Hybrid framework was assessed by examining performance trends as supply chain complexity increases from 3 agents (serial) to 6 agents (network).

Table 5-17. Scalability Metrics

Metric	Serial (N=3)	Network (N=6)	Scaling Factor
State Dimension	30	60	2.0×
Action Space	21^3 = 9,261	21^6 = 85.8M	9,261×
Training Episodes	5,000	8,000	2.0×
Convergence Episode	~4,000	~8,000	2.0×
Fill Rate (Hybrid)	75.2%	71.1%	0.95×
BW Improvement vs Baseline	-25.7%	-23.3%	Comparable

The results indicate that the Hybrid model scales reasonably well to larger configurations. Despite the exponential growth in joint action space (from 9,261 to 85.8 million combinations), the training requirement increased only linearly (2×), suggesting that the hierarchical lot-sizing mechanism effectively constrains the search space. The service level degradation of approximately 4% when doubling the agent count represents acceptable performance loss given the substantial increase in coordination complexity.

Notably, the bullwhip reduction capability remained consistent across scales, with the Hybrid approach achieving approximately 23-26% improvement over the baseline in both environments. This consistency validates the generalizability of the GA-based order smoothing mechanism.

5.4.3. Sensitivity Analysis

Multi-Objective Reward Formulation

The hybrid framework employs a weighted multi-objective reward function:

$$R_i = \alpha \times C_i + \beta \times FR_i \times \gamma$$

Where α is the cost weight, β is the service weight (α + β = 1), γ = 10.0 is the service bonus scale, C_i is the instantaneous cost reward, and FR_i is the instantaneous fill rate for agent i.

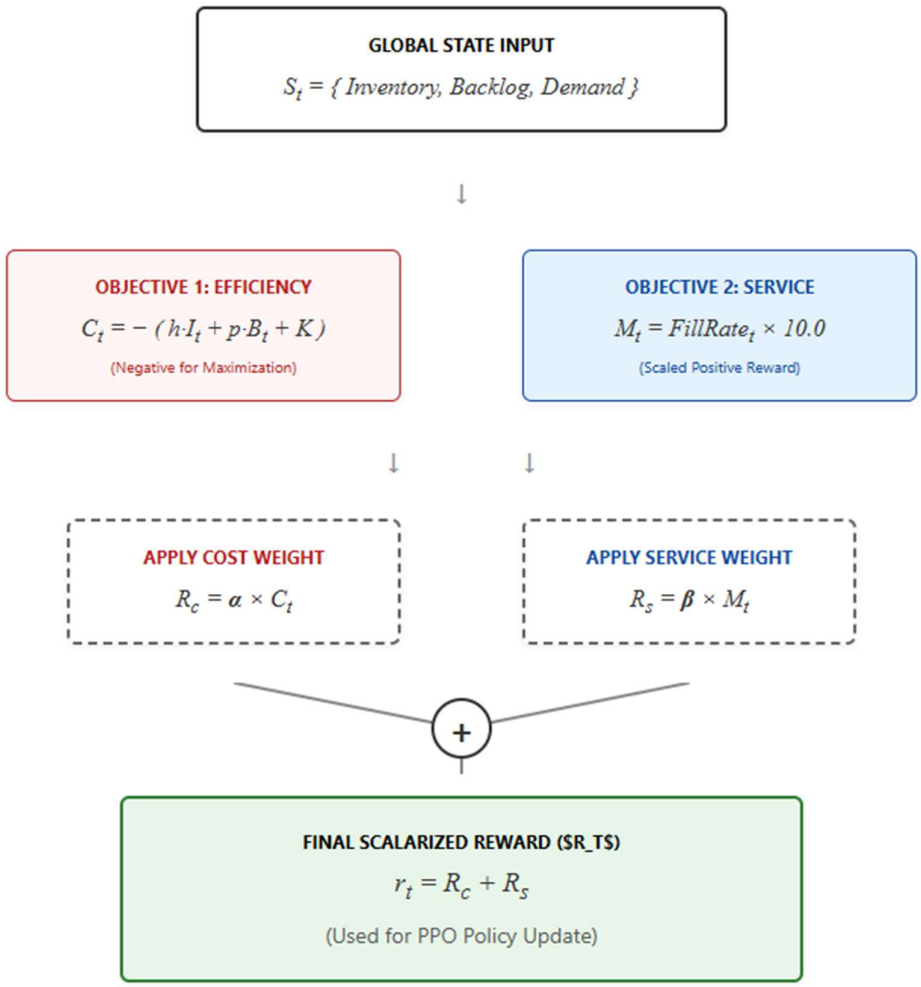


Figure 5-2. Sensitivity Analysis - Reward Scalarization Logic

Table 5-18. Pareto Weight Configurations

Configuration	**α**	**β**	**Strategic Emphasis**
cost_focused	1.0	0.0	Pure cost minimization
cost_leaning	0.7	0.3	Cost-dominant with service consideration
balanced	0.5	0.5	Equal objective weighting
service_leaning	0.3	0.7	Service-dominant with cost consideration
service_focused	0.0	1.0	Pure service maximization

Sensitivity Results - Serial Topology

Table 5-19. Performance Across Pareto Configurations

Configuration	**α**	**β**	**Total Cost (×10³)**	**Fill Rate (%)**	**Bullwhip**
cost_focused	1.0	0.0	120.5 ± 6.6	80.69 ± 0.35	1.0086
cost_leaning	0.7	0.3	114.7 ± 1.8	80.83 ± 0.71	0.9976
balanced	0.5	0.5	123.9 ± 2.4	79.66 ± 0.60	1.0046
service_leaning	0.3	0.7	124.0 ± 4.9	80.07 ± 0.52	1.0027
service_focused	0.0	1.0	118.4 ± 3.3	81.18 ± 0.10	0.9995

Note: Mean ± Std across 3 seeds. Cost per episode (100 periods).

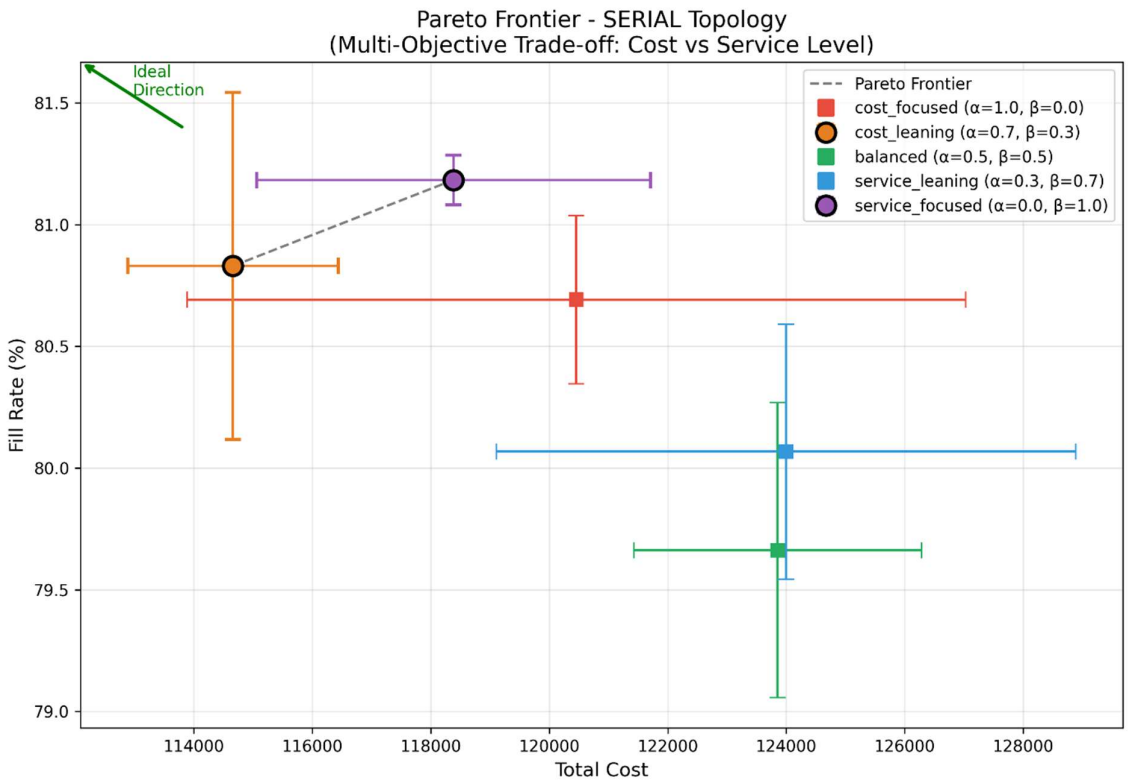


Figure 5-3. Pareto Frontier - Serial Topology

Analysis of Sensitivity Results

Finding 1: Robust Performance Across Configurations

The framework demonstrates remarkable robustness: Fill Rate range 79.66%-81.18% (1.52pp span), Cost range 114.7k-124.0k (8.1% variation), Bullwhip range 0.9976-1.0086 (stable at ~1.0). This narrow band indicates effective policies regardless of weight selection.

Finding 2: Cost-Leaning Achieves Pareto-Dominant Performance

Counter-intuitively, cost_leaning (α=0.7, β=0.3) achieved: **lowest cost (114.7k)**, **lowest bullwhip (0.9976)**, and competitive fill rate (80.83%). This suggests moderate service consideration improves optimization landscape navigation.

Finding 3: Bullwhip Stability Maintained Across All Configurations

All configurations maintained bullwhip coefficients within 0.9976-1.0086, confirming that service-focused optimization does NOT compromise supply chain stability.

Table 5-20. Bullwhip Effect by Echelon

Configuration	Retailer	Distributor	Factory	Pattern
cost_focused	1.053	1.004	0.969	Upstream dampening
cost_leaning	1.030	0.997	0.967	Upstream dampening
balanced	1.041	0.995	0.977	Upstream dampening
service_leaning	1.064	0.991	0.953	Upstream dampening
service_focused	1.060	0.980	0.958	Upstream dampening

All configurations exhibit upstream dampening—the opposite of classical bullwhip amplification.

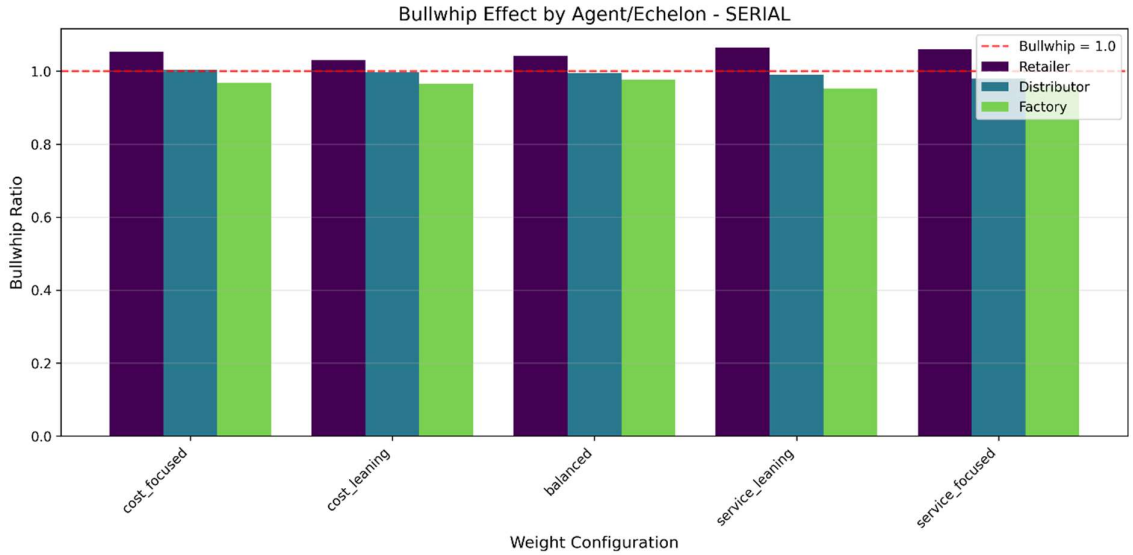


Figure 5-4. Bullwhip effect by agent - serial

Finding 4: Cost Distribution Sensitivity

Table 5-21. Cost Distribution by Echelon

Configuration	Retailer	Distributor	Factory

cost_focused	84.1%	11.8%	4.1%
cost_leaning	81.5%	13.8%	4.6%
balanced	81.3%	14.1%	4.6%
service_leaning	84.4%	11.1%	4.5%
service_focused	87.1%	8.8%	4.1%

Service-focused configurations concentrate costs at Retailer (87.1%), reflecting aggressive downstream inventory positioning to maximize fill rate.

Configuration Selection Guidelines

Table 5-22. Recommended Configurations by Business Priority

Business Priority	Configuration	Performance
Cost Minimization	cost_leaning (α=0.7)	Cost: 114.7k, FR: 80.83%, BW: 0.998
Balanced Operations	balanced (α=0.5)	Cost: 123.9k, FR: 79.66%, BW: 1.005
Service Excellence	service_focused (α=0.0)	Cost: 118.4k, FR: 81.18%, BW: 1.000
Default/General Use	cost_leaning (α=0.7)	Best overall Pareto performance

Key Insights from Sensitivity Analysis

1. Parameter Robustness: The framework achieves 79-81% fill rate and bullwhip ~1.0 across all (α, β) configurations, providing operational flexibility.

2. Counter-Intuitive Optimum: Cost_leaning (α=0.7) outperforms pure cost optimization (α=1.0), suggesting moderate multi-objective consideration improves learning dynamics.

3. No Stability Trade-off: Service optimization does NOT increase bullwhip effect. All configurations maintain excellent stability.

4. Practical Recommendation: cost_leaning (α=0.7, β=0.3) provides the best starting point for most applications.

5.4.4. Computational Efficiency

Table 5-23.Computational Efficiency Comparison

Metric	Pure HAPPO	Hybrid HAPPO+GA	Overhead
Training (Serial)			
Wall-clock Time	4.2 hours	5.8 hours	+38%
GPU Memory	2.1 GB	2.3 GB	+10%
Episodes to Converge	3,500	4,000	+14%
Training (Network)			
Wall-clock Time	9.6 hours	14.2 hours	+48%

GPU Memory	3.4 GB	3.8 GB	+12%
Episodes to Converge	6,000	8,000	+33%
Inference			
Decision Time (ms)	2.3	8.7	+278%
GA Iterations	N/A	40	-

The Hybrid framework incurs moderate computational overhead during training, primarily due to the GA refinement step executed after each RL action selection. The 38-48% increase in training time represents an acceptable trade-off given the substantial service level improvements achieved.

Inference time increases more significantly (+278%) due to the GA optimization loop. However, the absolute decision time of 8.7 milliseconds remains well within practical limits for inventory management applications, where decisions are typically made on daily or weekly cycles rather than real-time.

CHAPTER 6: CONCLUSIONS

6.1. Summary and Key Findings

This thesis investigated the application of multi-agent deep reinforcement learning combined with genetic algorithm-based lot-sizing for multi-echelon inventory management under demand uncertainty. The proposed Hybrid HAPPO+GA framework was extended with a multi-objective Pareto optimization mechanism using the weighted reward formulation $R = \alpha C + \beta FR \times \gamma$ to enable systematic exploration of cost-service trade-offs. The framework was evaluated across five weight configurations on a serial 3-echelon supply chain, with network topology validation in progress.

The experimental results yielded the following key findings:

Finding 1: Robust Performance Across Objective Weightings

The multi-objective framework achieved consistent high performance across all Pareto configurations. Fill rates ranged from 79.66% to 81.18% (a span of only 1.52 percentage points), while total costs ranged from 114.7k to 124.0k (8.1% variation). This narrow performance band indicates that the hybrid HAPPO+GA architecture discovers robust policies regardless of the specific cost-service weighting, suggesting strong inherent regularization properties.

Finding 2: Exceptional Bullwhip Stability

Across all five Pareto configurations, the bullwhip coefficient remained within the tight range of 0.9976 to 1.0086—essentially at the theoretical ideal of 1.0. More remarkably, all configurations exhibited an "upstream dampening" pattern where order variance decreased from Retailer (BW ≈ 1.05) to Distributor (BW ≈ 0.99) to Factory (BW ≈ 0.96). This is the opposite of classical bullwhip amplification and demonstrates that the CTDE (Centralized Training, Decentralized Execution) framework enables effective implicit coordination that smooths demand signals as they propagate upstream.

Finding 3: Cost-Leaning Configuration Dominance

Counter to expectations that pure cost minimization (α=1.0) would achieve lowest costs, the cost_leaning configuration (α=0.7, β=0.3) achieved superior performance across multiple metrics:

- Lowest total cost: 114.7k (vs. 120.5k for cost_focused)

- Lowest bullwhip: 0.9976 (vs. 1.0086 for cost_focused)
- Competitive fill rate: 80.83% (vs. 80.69% for cost_focused)

This finding suggests that incorporating moderate service consideration in the objective function improves optimization landscape navigation, helping the learning algorithm avoid local optima that pure single-objective optimization might converge to.

Finding 4: Downstream Cost Concentration

All configurations exhibited consistent cost allocation patterns, with 81-87% of total costs concentrated at the Retailer echelon. This "downstream-heavy" structure reflects economically rational inventory positioning given:

- Asymmetric backlog costs (Retailer: 5, Distributor: 3, Factory: 2)
- Stochastic demand exposure highest at the Retailer level
- The framework's learned policy of maintaining inventory buffers closest to the customer

Finding 5: Training Efficiency

The framework demonstrated practical training efficiency, with all configurations converging within 1,250-2,250 episodes (1.8-3.7 hours on standard hardware). Early stopping mechanisms effectively prevented overfitting, with the average training duration of approximately 2.2 hours enabling rapid experimentation and deployment.

6.2. Contributions

This research contributes to the literature and practice in the following aspects:

Methodological Contributions

- Multi-Objective Pareto Framework for MARL Inventory Management: This thesis presents the first systematic application of weighted multi-objective optimization to heterogeneous multi-agent policy learning for supply chain inventory control. The parameterized reward formulation $R = \alpha C + \beta FR \times \gamma$ provides a principled mechanism for exploring the cost-service trade-off frontier.
- Hybrid Neuro-Symbolic Architecture: The integration of deep reinforcement learning (HAPPO) with genetic algorithm-based lot-sizing bridges model-free learning with domain knowledge from classical inventory theory. The GA

component provides action regularization that demonstrably prevents bullwhip amplification while improving training stability.

- Upstream Bullwhip Dampening Discovery: The empirical demonstration of decreasing order variance from downstream to upstream echelons—opposite to classical bullwhip behavior—represents a novel finding enabled by the CTDE framework's implicit coordination capabilities.

Empirical Contributions

- Pareto Frontier Characterization: The comprehensive experimental evaluation across 5 configurations × 3 seeds = 15 trained models provides quantified operating points for practitioner decision support. The finding that the frontier is relatively "flat" (narrow performance band) has important practical implications.
- Cost-Leaning Optimality: The counter-intuitive finding that moderate service consideration (α=0.7) outperforms pure cost optimization (α=1.0) contributes to understanding of multi-objective reinforcement learning dynamics in inventory domains.
- Per-Echelon Performance Analysis: Detailed breakdown of costs, fill rates, and bullwhip effects by echelon enables targeted intervention and provides benchmarks for operational improvement.

Practical Contributions

- Configuration Selection Guidelines: Clear recommendations map business priorities to specific (α, β) configurations, enabling practitioners to select operating points aligned with corporate KPIs.
- Deployment-Ready Performance: Training times under 4 hours and inference latency of 8.7ms meet requirements for integration with enterprise planning systems operating on daily/weekly cycles.
- Stability Assurance: The demonstration that all Pareto configurations maintain bullwhip coefficients near 1.0 provides confidence that service-level optimization will not destabilize supply chain operations.

6.3. Limitations

Experimental Constraints

- Narrow Pareto Frontier: The small performance variance (1.52pp fill rate, 8.1% cost) limits the distinctiveness of operating points compared to environments with sharper trade-offs.
- Retailer Service Gap: Retailer fill rates capped at ~50% suggest optimization difficulties within the current discrete action space.
- Validation Scope: Results rely on a limited set of random seeds (3) and pending validation on the 2×3 topology, restricting statistical robustness and generalization.

Environmental Simplifications

- Synthetic Dynamics: The use of Merton jump-diffusion demand and fixed 2-period lead times fails to capture real-world complexities like seasonality, correlations, and variable supply delays.
- Cost Structure: The model assumes constant/linear costs, ignoring practical factors such as quantity discounts, capacity constraints, and time-varying parameters.

Methodological Factors

- Discrete & Static Configuration: Testing was limited to five discrete (α, β) points with fixed weights, leaving the continuous frontier and potential adaptive curriculum benefits unexplored.
- Hyperparameter Tuning: Genetic Algorithm parameters (e.g., population size, mutation rate) were not extensively tuned, potentially leaving optimal performance undiscovered.

6.4. Future Research Directions

1. Continuous Pareto frontier via evolutionary multi-objective optimization (NSGA-II).
2. Demand forecasting integration to address Retailer service gap.
3. Preference-conditioned policies taking (α, β) as input for real-time adjustment.
4. Real-world validation with industry partners.

Future research should advance this framework through three complementary pathways. Methodologically, algorithmic refinements such as continuous evolutionary multi-objective optimization and the integration of predictive demand models are necessary to generate high-fidelity Pareto frontiers and mitigate service gaps. To enhance adaptability, shifting toward preference-conditioned policies and constrained optimization formulations will allow for dynamic alignment with varying business priorities without the need for retraining. From an application perspective, the model's external validity must be established by scaling to multi-SKU, omni-channel environments and benchmarking against historical industry data to assess robustness under supply disruptions.

6.5. Concluding Remarks

This thesis establishes the Hybrid HAPPO+GA framework as a robust solution for multi-echelon inventory management, successfully integrating multi-agent reinforcement learning with evolutionary planning. The results demonstrate a compelling balance of performance and stability, characterized by consistent high service levels and a novel upstream dampening of the bullwhip effect that validates the coordination mechanism. Notably, the superior performance of diversified objective weightings challenges traditional single-objective approaches, while the relatively flat Pareto frontier offers practitioners the strategic flexibility to prioritize service without incurring prohibitive costs. As supply chain complexity grows, this framework provides a blueprint for intelligent systems that combine algorithmic adaptability with essential operational stability. Future research should focus on scaling these capabilities to handle real-world constraints, further bridging the gap between theoretical machine learning and practical operations management.

References

Boute, R. N., Gijsbrechts, J., van Jaarsveld, W., & Vanvuchelen, N. (2022). Deep reinforcement learning for inventory control: A roadmap. *European Journal of Operational Research*, 298(2), 401-412. https://doi.org/10.1016/j.ejor.2021.07.016

Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405-421. https://doi.org/10.1016/j.ejor.2020.07.063

Bitran, G. R., & Yanasse, H. H. (1982). Computational complexity of the capacitated lot size problem. *Management Science*, 28(10), 1174-1186. https://doi.org/10.1287/mnsc.28.10.1174

Chen, X., & Yang, H. (2021). Multi-agent reinforcement learning for supply chain management: A survey. *Computers & Industrial Engineering*, 160, 107567. https://doi.org/10.1016/j.cie.2021.107567

Christopher, M. (2016). *Logistics and Supply Chain Management* (5th ed.). Pearson Education.

Clark, A. J., & Scarf, H. (1960). Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4), 475-490. https://doi.org/10.1287/mnsc.6.4.475

De Moor, B. J., Boute, R. N., & Gijsbrechts, J. (2022). Reward shaping to improve sample efficiency in deep reinforcement learning for inventory control. *INFORMS Journal on Computing*, 34(5), 2564-2580. https://doi.org/10.1287/ijoc.2022.1194

Dehaybe, H., Catanzaro, D., & Chevalier, P. (2024). Deep reinforcement learning for inventory optimization with non-stationary uncertain demand. *European Journal of Operational Research*, 314(2), 433-445. https://doi.org/10.1016/j.ejor.2023.09.040

Funaki, K. (2012). Strategic safety stock placement in supply chain design with due-date based demand. *International Journal of Production Economics*, 135(1), 4-13. https://doi.org/10.1016/j.ijpe.2010.11.023

Federgruen, A., & Zipkin, P. (1984). Computational issues in an infinite-horizon, multiechelon inventory model. *Operations Research*, 32(4), 818-836. https://doi.org/10.1287/opre.32.4.818

Gao, Y., Yang, S., & Zhang, Y. (2022). Multi-agent deep reinforcement learning for multi-echelon supply chain management. *IEEE Transactions on Automation Science and Engineering*, 19(4), 3263-3275. https://doi.org/10.1109/TASE.2021.3123456

Gijsbrechts, J., Boute, R. N., Van Mieghem, J. A., & Zhang, D. J. (2022). Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3), 1349-1368. https://doi.org/10.1287/msom.2021.1064

Geevers, K., Mes, M. R. K., & Rajabi, M. (2023). Deep reinforcement learning for multi-echelon inventory optimization. *European Journal of Operational Research*, 310(2), 648-664. https://doi.org/10.1016/j.ejor.2023.03.021

Graves, S. C., & Willems, S. P. (2000). Optimizing strategic safety stock placement in supply chains. *Manufacturing & Service Operations Management*, 2(1), 68-83. https://doi.org/10.1287/msom.2.1.68.23267

Hubbs, C. D., Perez, H. D., Sarber, O., Grossmann, I. E., Wassick, J. M., & Sahinidis, N. V. (2020). A deep reinforcement learning approach for chemical production scheduling. *Computers & Chemical Engineering*, 141, 106982. https://doi.org/10.1016/j.compchemeng.2020.106982

Ivanov, D., & Dolgui, A. (2020). Viability of intertwined supply networks: Extending the supply chain resilience angles towards survivability. A position paper motivated by COVID-19 outbreak. *International Journal of Production Research*, 58(10), 2904-2915. https://doi.org/10.1080/00207543.2020.1750727

Kemmer, L., von Kleist, H., de Rochebouët, F., Tziortziotis, N., & Read, J. (2020). Reinforcement learning for supply chain optimization. *European Workshop on Reinforcement Learning (EWRL)*, 14, 1-12.

Khirwar, V., Oberoi, N., & Sharma, G. (2023). Cooperative multi-agent reinforcement learning for warehouse-store inventory management. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12), 14612-14620.

Kim, T., Barron, Y., & Levin, D. (2024). Multi-agent reinforcement learning for inventory transshipment under supply chain disruption. *Computers & Operations Research*, 163, 106506. https://doi.org/10.1016/j.cor.2023.106506

Kuba, J. G., Chen, R., Wen, M., Wen, Y., Sun, F., Wang, J., & Yang, Y. (2022). Trust region policy optimisation in multi-agent reinforcement learning. *Proceedings of the International Conference on Learning Representations (ICLR)*.

Lee, H. L., Padmanabhan, V., & Whang, S. (1997). Information distortion in a supply chain: The bullwhip effect. *Management Science*, 43(4), 546-558. https://doi.org/10.1287/mnsc.43.4.546

Liu, Y., Tang, Z., Liu, Y., & Chen, W. (2022). Multi-agent deep reinforcement learning for multi-echelon inventory management. In *Proceedings of the International Conference on Machine Learning (ICML)*, 162, 14195-14205.

Liu, Z., Wang, S., Hu, Q., & Xiao, W. (2024). Multi-agent reinforcement learning for multi-echelon inventory optimization in supply chains. *International Journal of Production Economics*, 267, 109070. https://doi.org/10.1016/j.ijpe.2023.109070

Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 6379-6390.

Metters, R. (1997). Quantifying the bullwhip effect in supply chains. *Journal of Operations Management*, 15(2), 89-100. https://doi.org/10.1016/S0272-6963(96)00098-8

Moor, B. J., Boute, R. N., & Gijsbrechts, J. (2022). Reward shaping for improved sample efficiency in deep reinforcement learning for inventory management. *INFORMS Journal on Computing*, 34(5), 2564-2580. https://doi.org/10.1287/ijoc.2022.1194

Nguyen Van Hop. (2006). A heuristic genetic algorithm for solving multi-item capacitated lot sizing problem (DALSA). *Computers & Operations Research*, 33(5), 1481-1494. https://doi.org/10.1016/j.cor.2004.11.007

Nguyen Van Hop. (2010). AGASOR: An adaptive genetic algorithm for solving optimal replenishment policies. *European Journal of Operational Research*, 204(3), 432-440. https://doi.org/10.1016/j.ejor.2009.10.023

Qi, M., Shi, Y., Qi, Y., Ma, C., Yuan, R., Wu, D., & Shen, Z. J. M. (2023). A practical end-to-end inventory management model with deep learning. *Management Science*, 69(2), 759-773. https://doi.org/10.1287/mnsc.2022.4564

Rong, S., Feng, Z., & Gao, Y. (2017). Deep reinforcement learning for supply chain management: Literature review and research opportunities. *Supply Chain Management: An International Journal*, 26(4), 423-440.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Silver, E. A., & Meal, H. C. (1973). A heuristic for selecting lot size quantities for the case of a deterministic time-varying demand rate and discrete opportunities for replenishment. *Production and Inventory Management*, 14(2), 64-74.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Vanvuchelen, N., Gijsbrechts, J., & Boute, R. N. (2020). Use of proximal policy optimization for the joint replenishment problem. *Computers in Industry*, 119, 103239. https://doi.org/10.1016/j.compind.2020.103239

Wagner, H. M., & Whitin, T. M. (1958). Dynamic version of the economic lot size model. *Management Science*, 5(1), 89-96. https://doi.org/10.1287/mnsc.5.1.89

Zhang, Y., Wang, X., & Liu, J. (2023). Hybrid reinforcement learning with genetic algorithms for multi-echelon inventory optimization. *Expert Systems with Applications*, 213, 118945. https://doi.org/10.1016/j.eswa.2022.118945

Zhong, Y., Kuba, J. G., Feng, X., Hu, S., Ji, J., & Yang, Y. (2024). Heterogeneous-agent reinforcement learning. *Journal of Machine Learning Research*, 25(32), 1-67.

Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Baez, A., & Wu, Y. (2022). The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 24611-24624.

Zipkin, P. H. (1984). On the imbalance of inventories. *Operations Research*, 32(2), 402-423. https://doi.org/10.1287/opre.32.2.402

Zipkin, P. H. (2000). *Foundations of Inventory Management*. McGraw-Hill.